

A PROJECT-DRIVEN MINI-BOOK & ROADMAP

Go, the 20%

The critical fifth of the Go language, learned by building real things, for developers who already know Python, JavaScript or TypeScript, and who want to prove they can still build software without a model in the loop.

Four projects · a CLI, a concurrent link checker, a web service with a UI and SQLite, an uptime monitor

Six weeks · about eight to ten hours a week, milestone by milestone

Zero AI assistance · while building, by contract

Go 1.22 or newer · current release 1.27, September 2026

01

taskr

CLI + JSON

02

linkcheck

goroutines + channels

03

shelf

HTTP API + HTML + SQLite

04

pulse

workers + live dashboard

Contents

Before you start · How to use this book	3
PART I · ORIENTATION	6
Chapter 1 · Why Go, and what the 20% is	7
Chapter 2 · Mental model shifts from Python and TypeScript	9
Chapter 3 · The toolchain in twenty minutes	12
Chapter 4 · The AI-free method	16
PART II · THE CORE LANGUAGE	18
Chapter 5 · Syntax essentials	19
Chapter 6 · Data: structs, methods, pointers, slices, maps, strings	24
Chapter 7 · Errors as values	29
Chapter 8 · Interfaces	32
Chapter 9 · Generics, the 20% version	35
Chapter 10 · Testing	38
Project 1 · taskr , a task-tracking CLI	41
PART III · CONCURRENCY	45
Chapter 11 · Goroutines and the scheduler	46
Chapter 12 · Channels and select	48
Chapter 13 · Sharing memory safely: sync and atomic	51
Chapter 14 · Cancellation with context	54
Chapter 15 · The five patterns that cover 80% of concurrent code	56
Chapter 16 · Pitfalls and the race detector	60
Project 2 · linkcheck , a concurrent link checker	62
PART IV · BUILDING REAL SERVICES	65
Chapter 17 · The net/http server	66
Chapter 18 · An HTML frontend: templates, static files, embed	70
Chapter 19 · Persistence with database/sql and SQLite	73
Chapter 20 · Configuration, logging and shipping one binary	77
Project 3 · shelf , a reading-list API with a web UI	80
PART V · CAPSTONE AND ROADMAP	84
Project 4 · Capstone · pulse , a local uptime monitor with a live dashboard	85
Chapter 21 · The six-week roadmap	89
Chapter 22 · Beyond the 20%	91
APPENDICES · REFERENCE	93
Appendix A · The idiom cheat sheet	94
Appendix B · Error message decoder	97
Appendix C · Self-assessment checklists	100

How to use this book

This is a mini-book, a guideline, and a roadmap in one. It is written for a developer with several years of Python, JavaScript and TypeScript who wants the 20% of Go that does 80% of real work, and who wants to learn it by *building*, not by solving puzzles.

What this book is (and is not)

It **is** a deliberately narrow, deep pass over the parts of Go you will use every day: the toolchain, the type system, errors, interfaces, slices and maps, goroutines and channels, `net/http`, testing, and a small amount of persistence. Each concept is introduced with a short explanation, a mental-model note for people coming from Python/TypeScript, an idiomatic code fragment, and the gotchas that actually bite.

It **is not** a language reference. It skips reflection, `cgo`, assembly, the details of the garbage collector, and most of the standard library on purpose. When a topic is intentionally left out, the book says so, and Chapter 22 tells you where to go when you need it later.

The book is organized around four projects of increasing size. The chapters exist to serve the projects, not the other way around. If you only read, you will forget; if you only build, you will cargo-cult. Do both, in the order given.

The project ladder

#	PROJECT	WHAT YOU BUILD	CORE SKILLS EXERCISED	TIME BOX
1	<code>taskr</code>	A task-tracking CLI that persists to a JSON file	Modules, structs, slices, maps, errors, JSON, files, <code>flag</code> , tests	6–8 h
2	<code>linkcheck</code>	A concurrent link checker with a worker pool and timeouts	Goroutines, channels, <code>select</code> , <code>sync</code> , <code>context</code> , <code>net/http</code> client, race detector	6–8 h
3	<code>shelf</code>	A reading-list REST API with an HTML/JS frontend and SQLite	<code>net/http</code> server, routing, JSON APIs, middleware, templates, <code>embed</code> , <code>database/sql</code> , <code>httptest</code>	10–14 h
4	<code>pulse</code>	A local uptime monitor with background workers and a live dashboard	Everything above, combined: tickers, cancellation, persistence, server-sent events, graceful shutdown	10–14 h

Every project is small enough to finish in a few evenings and real enough that you would not be embarrassed to put it on GitHub. None of them is a toy problem. Each one has milestones, acceptance criteria, and a definition of done so you always know whether you are finished.

The contract

You said you want to do this without AI assistance, to rebuild the skills that have atrophied. That goal is achievable, but only if the rule is explicit and you keep it under pressure. Here is the contract this book assumes:

1. **While building a project, no AI at all.** No autocomplete from a model, no chat window, no “just explain this error”. Your allowed sources are: this book, the official docs on `pkg.go.dev`, `go doc` in your terminal, the Go Tour, Go by Example, and your own experiments.
2. **Type every code fragment in this book by hand.** Never copy-paste. Typing is where the syntax becomes muscle memory and where you notice details you would otherwise skim.
3. **When you are stuck, you have a 25-minute struggle budget** before you consult the docs (not AI). Use that time to read the compiler error slowly, add `fmt.Printf`, and shrink the problem. Most Go errors are precise enough that the fix is in the message.
4. **Only after a project is fully done and committed**, you may optionally use an AI as a *code reviewer*: paste your code, ask what is non-idiomatic and why. Read the answer, close the window, and make the changes yourself. Never accept generated code into a learning project.
5. **Keep a `LEARNING.md` in every project.** One line per session: what you built, what confused you, what you’d do differently. This is the single highest-leverage habit for retention.

Chapter 4 expands on this method, including how to debug and how to get unstuck without a model.

How the chapters are marked

MENTAL SHIFT · COMING FROM PYTHON / TYPESCRIPT

Notes like this translate a Go idea into terms you already know from Python or TypeScript, and point out where the mapping is misleading.

GOTCHA

Notes like this describe a mistake that experienced developers make when they start Go. Each one has cost someone an afternoon.

DO THIS NOW · NO AI, JUST YOU

Notes like this ask you to stop reading and do something small in your terminal right now. They take two to ten minutes and are not optional.

RULE OF THUMB

Notes like this compress a design decision into a rule you can apply without thinking, with the reasoning underneath it.

VERSION NOTE

Notes like this mark features that depend on a specific Go release. The book assumes Go 1.22 or newer; the current release as of September 2026 is Go 1.27.

Suggested pace

The full path is about six weeks at roughly eight to ten hours a week. Chapter 21 gives a week-by-week plan and a slower, milestone-based alternative if your time is irregular. A sensible minimum for a productive session is 60 minutes; shorter sessions rarely get past the warm-up.

Read Part I in one sitting. Then alternate: read a chapter, immediately apply it to the milestone that uses it. Do not read ahead of the milestone you are on; Go is easy to *read* and easy to forget.

PART I

Orientation

What the 20% of Go actually is, how Go will feel different from Python and TypeScript, how the toolchain works, and how to learn it without a model doing the thinking for you.

Chapter 1 · Why Go, and what the 20% is

Chapter 2 · Mental model shifts from Python and TypeScript

Chapter 3 · The toolchain in twenty minutes

Chapter 4 · The AI-free method

CHAPTER 1

Why Go, and what the 20% is

Go is a small language with a large standard library and an opinionated toolchain. That combination is exactly why the 80/20 rule works better for Go than for almost any other mainstream language.

What Go is optimized for

Go was designed at Google in 2007–2009 by Robert Griesemer, Rob Pike and Ken Thompson to solve a specific set of problems: long compile times, dependency hell, and the difficulty of writing correct concurrent server software in C++ and Java. Everything in the language follows from those goals.

- **Fast compilation and a single static binary.** `go build` produces one executable with no runtime to install. Deploying a Go service is copying a file.
- **Concurrency as a language feature.** Goroutines (cheap, runtime-scheduled threads) and channels (typed queues) are built into the syntax, not bolted on as a library. This is the “native multithreading” you have heard about, and it is the single most valuable thing Go will teach you.
- **A small surface area.** Go has 25 keywords. There is one loop construct. There are no classes, no inheritance, no exceptions, no operator overloading, no implicit conversions. Learning it is mostly learning to *stop* reaching for features.
- **Batteries included.** The standard library ships a production-grade HTTP server and client, JSON, templating, testing, cryptography, and SQL access. Most Go services have a handful of dependencies, not hundreds.
- **Tooling that removes arguments.** `gofmt` formats code one way. `go vet` finds bugs. `go test` runs tests. There is no configuration to argue about.

The result is a language where the idiomatic way to do something is usually the only reasonable way, which means the 20% you need is unusually well-defined.

The 80/20 map

Here is the whole book on one page. Tier A is what you master. Tier B is what you need to recognize and can look up. Tier C is what you consciously skip until you have a concrete reason.

TIER	TOPIC	WHY IT'S IN THIS TIER
A • MASTER	Modules, packages, <code>go</code> commands, <code>gofmt</code> / <code>vet</code>	Everything else sits on top of this
A • MASTER	Variables, zero values, <code>:=</code> , <code>const</code> , functions with multiple returns, <code>for</code> , <code>switch</code> , <code>defer</code>	Core syntax; used in every file
A • MASTER	Structs, methods, pointers vs values, embedding	Go's replacement for classes

TIER	TOPIC	WHY IT'S IN THIS TIER
A • MASTER	Slices and maps, including the slice memory model	The two data structures of Go; slices bite Python devs
A • MASTER	Errors as values: wrapping, <code>errors.Is</code> / <code>As</code> , sentinel and custom errors	The idiom that shapes every function signature
A • MASTER	Interfaces: implicit satisfaction, <code>io.Reader</code> / <code>io.Writer</code> , type switches	How Go achieves polymorphism and testability
A • MASTER	Goroutines, channels, <code>select</code> , <code>sync.WaitGroup</code> , <code>sync.Mutex</code> , <code>context</code>	Go's standout capability
A • MASTER	<code>net/http</code> server and client, JSON, middleware, <code>httptest</code>	Most Go jobs are services
A • MASTER	<code>testing</code> : table-driven tests, subtests, <code>-race</code> , <code>-cover</code>	Tests are how Go programmers debug
B • RECOGNIZE	Generics (type parameters and constraints)	Useful for helpers; rarely needed in application code
B • RECOGNIZE	<code>html/template</code> , <code>embed</code> , <code>database/sql</code> , <code>log/slog</code> , <code>flag</code> , <code>os/signal</code>	Standard-library pieces you assemble services from
B • RECOGNIZE	Strings, runes and bytes; <code>strings</code> , <code>strconv</code> , <code>bufio</code>	You use them constantly but they are simple
B • RECOGNIZE	<code>errgroup</code> , <code>sync.Once</code> , <code>atomic</code> , <code>sync.RWMutex</code>	Second-tier concurrency tools
B • RECOGNIZE	Iterators (<code>iter.Seq</code> , range-over-func), generic methods	Newer language features; read them, rarely write them at first
C • SKIP FOR NOW	Reflection, <code>unsafe</code> , <code>cgo</code> , assembly, build constraints, <code>go:generate</code>	Powerful, rarely needed, easy to misuse
C • SKIP FOR NOW	GC tuning, <code>pprof</code> , escape analysis, PGO	Learn when you have a measured performance problem
C • SKIP FOR NOW	<code>gRPC</code> , <code>protobuf</code> , large web frameworks, ORMs	Ecosystem, not language; the standard library is enough here

Notice what is not in Tier A: frameworks. In Go, the standard library `net/http` package is the framework for most services, and learning it directly means you will understand every third-party router in an afternoon later.

What Go is not good at

Being honest about this saves frustration. Go is verbose about errors, which is deliberate but tiring at first. It has no sum types or pattern matching, so modelling “one of these shapes” is done with interfaces and type switches. Its generics are intentionally limited. Its dependency ecosystem is smaller than npm or PyPI, and the community is skeptical of dependencies in general. For scripting, data science and rapid UI prototyping you will still reach for Python or TypeScript. For network services, CLIs, infrastructure tools and anything concurrent, Go is very hard to beat.

CHAPTER 2

Mental model shifts from Python and TypeScript

Your four years of experience are an asset, but some of your instincts will actively mislead you. This chapter names the eight shifts up front so you can recognize them when they happen.

Shift 1: Errors are values, not events

Python raises; JavaScript throws; Go *returns*. A function that can fail returns an error as its last result, and the caller checks it immediately:

```
f, err := os.Open("config.json")
if err != nil {
    return fmt.Errorf("open config: %w", err)
}
defer f.Close()
```

There is no `try/catch`. The `if err != nil` block appears everywhere, and you will initially find it repetitive. Within a week it becomes the thing you like most about Go: every failure point is visible at the call site, and control flow is never hijacked by an exception three frames up. `panic` exists but is for programmer bugs (nil dereference, index out of range), not for expected failures like a missing file or a bad HTTP status.

Shift 2: No classes, no inheritance, no `this`

Go has structs (data) and methods (functions attached to a type). It has no class hierarchies. Code reuse comes from **composition** (embedding one struct in another) and **interfaces** (a set of method signatures that any type can satisfy without declaring it). You will stop asking “what should this inherit from?” and start asking “what behaviour does this need to expose?”

Shift 3: Static typing that mostly stays out of the way

TypeScript’s type system is structural, gradual, and enormously expressive. Go’s is structural for interfaces, nominal for everything else, and deliberately small. There are no union types, no optional chaining, no `any`-by-default escape hatch (there is `any`, but using it is a code smell). Type inference with `:=` means you rarely write types for local variables, but you always write them for function signatures and struct fields. The compiler is strict about it: an unused variable or import is a compile *error*, not a warning.

Shift 4: Zero values instead of `undefined` / `None`

Every type in Go has a zero value that is ready to use: `0`, `""`, `false`, `nil` for pointers, slices, maps, channels, functions and interfaces. A declared-but-unassigned struct is a struct full of zero values, not `undefined`. This lets you write `var buf bytes.Buffer` and use it immediately. The flip side is that `nil` maps panic on write and `nil` pointers panic on dereference, so you must know which types need `make` or `&T{}` before use (Chapter 6).

Shift 5: Value semantics by default

In Python and JavaScript, objects are references: passing a dict to a function lets the function mutate the caller's dict. In Go, structs, arrays and basic types are copied when assigned or passed. If you want the callee to mutate the caller's value, you pass a pointer (`&x`) explicitly. Slices, maps and channels are the exception: they are small headers that refer to shared underlying data, which is why Chapter 6 spends so long on slices.

Shift 6: Concurrency is synchronous code running in parallel

There is no `async` / `await` and no event loop you have to feed. You write ordinary blocking code, and you run it concurrently by prefixing a call with `go`. Goroutines communicate through channels or share memory guarded by a mutex. Blocking a goroutine is cheap and normal; the runtime multiplexes thousands of them onto a handful of OS threads. Coming from JavaScript this feels like a superpower; coming from Python it feels like the GIL was a bad dream.

Shift 7: The formatter is not negotiable

`gofmt` rewrites your code into the canonical layout: tabs, brace placement, spacing, import grouping. Every Go codebase on earth looks the same. Set your editor to format on save and never think about style again.

Shift 8: Explicit beats clever

Go programmers write plain code. Loops instead of `map` / `filter` chains. Early returns instead of nested conditionals. A 15-line function instead of a one-liner with three closures. The language rewards this by compiling it fast and making it trivial to debug. Resist the urge to “make it elegant”; the elegant version is usually the boring one.

Concept translation table

YOU KNOW THIS IN PYTHON / TYPESCRIPT	IN GO YOU WRITE	NOTES
<code>class</code> with methods	<code>type T struct{...} + func (t *T) Method()</code>	Methods are declared outside the struct
Inheritance / <code>extends</code>	Struct embedding + interfaces	Composition, not hierarchy
<code>interface</code> (TS) / duck typing (Py)	<code>type I interface{ Method() }</code>	Satisfied implicitly, checked at compile time

YOU KNOW THIS IN PYTHON / TYPESCRIPT	IN GO YOU WRITE	NOTES
try/except, throw	val, err := f(); if err != nil { ... }	Errors are ordinary return values
None / null / undefined	nil (pointers, slices, maps, chans, funcs, interfaces) or the zero value	Value types are never nil
list, Array<T>	[]T (slice)	Dynamic length; shares backing storage
dict, Map<K,V>, object-as-map	map[K]V	Unordered; must be <code>make</code> d before writing
Tuple return (a, b)	func f() (A, B)	Native multiple return values
*args / ...rest	func f(xs ...int)	Variadic; pass a slice with <code>f(s...)</code>
Lambda / arrow function	func(x int) int { return x*2 }	Closures capture by reference
async/await, asyncio, Promises	go f(), channels, sync.WaitGroup	Blocking code, run concurrently
with open(...) / try/finally	defer f.Close()	Runs when the enclosing <i>function</i> returns
Modules / packages / import	Package per directory; import "path"	Exported names start with a capital letter
pip, npm, package.json	go.mod, go get, go mod tidy	Built into the go tool
pytest, jest	go test, files ending in _test.go	In the standard library
Generics <T>	func F[T any](x T)	Present since 1.18; simpler than TS generics
enum	const block with iota	No real enums; conventions instead
f-strings / template literals	fmt.Sprintf("%s is %d", name, age)	Format verbs: %v %d %s %q %T %+v

Keep this table nearby for the first two weeks. After that you will not need it.

CHAPTER 3

The toolchain in twenty minutes

Go ships one binary, `go`, that builds, tests, formats, vets, documents and manages dependencies. There is no separate package manager, test runner or bundler to configure. Learn these commands once and you have the whole workflow.

Install and verify

Download the installer for your OS from <https://go.dev/dl/> and follow the one-screen instructions. Then, in a fresh terminal:

```
go version          # go version go1.27.0 darwin/arm64 (or linux/amd64, windows/amd64)
go env GOPATH       # where downloaded modules and installed tools live
```

VERSION NOTE

This book assumes Go 1.22 or newer. Go 1.22 changed loop-variable semantics and added the pattern-based HTTP router used in Part IV; Go 1.21 added the `slices`, `maps` and `log/slog` packages and the `min`/`max`/`clear` builtins. Install the current release (1.27 at the time of writing) and you have all of it. Go's compatibility promise means code written for 1.22 keeps compiling.

Editor

Install VS Code with the official **Go** extension (by the Go team), or use GoLand, Neovim with `gopls`, or Zed. All of them talk to `gopls`, the Go language server, which provides completion, go-to-definition, inline diagnostics and formatting. Two settings matter:

1. **Format on save** must be on. The extension uses `gofmt` (and `goimports`, which also adds and removes import lines for you).
2. **Turn off AI autocomplete** in this workspace. Copilot-style completion is precisely the dependency you are trying to break. The Go extension's own completions come from `gopls` and are fine: they show you what exists, not what to write.

Modules: the unit of versioning

A **module** is a tree of packages with a `go.mod` file at its root. A **package** is a directory of `.go` files that share a `package` clause. One module usually equals one repository. Create one:

```
mkdir taskr && cd taskr
go mod init github.com/yourname/taskr # module path; doubles as your import prefix
```

This writes `go.mod`:

```
module github.com/yourname/taskr

go 1.27
```

The module path does not have to exist on GitHub for local work; it is just a name. Using a URL-shaped path is a convention that makes publishing trivial later.

The commands you will run every day

COMMAND	WHAT IT DOES	WHEN
<code>go run .</code>	Compile and run the package in the current directory	Fast feedback while developing
<code>go build -o bin/taskr .</code>	Produce a binary	When you want an executable to keep
<code>go test ./...</code>	Run all tests in the module (<code>./...</code> means “this directory and below”)	Constantly
<code>go test -run TestAdd -v ./internal/store</code>	Run one test, verbosely, in one package	Debugging a failure
<code>go test -race ./...</code>	Run tests with the data race detector	Always for concurrent code
<code>go test -cover ./...</code>	Show coverage per package	Sanity check, not a target
<code>go vet ./...</code>	Static analysis for likely bugs (printf mismatches, unreachable code, copies of locks)	Before every commit
<code>go fmt ./...</code>	Format all files (your editor already does this)	If you edited outside the editor
<code>go get github.com/x/y@latest</code>	Add or upgrade a dependency	Rarely
<code>go mod tidy</code>	Add missing and remove unused requirements; update <code>go.sum</code>	After adding/removing imports
<code>go doc strings.Builder</code>	Print documentation for a package, type or function	Instead of searching the web
<code>go doc -all encoding/json \ less</code>	Full docs for a package	Learning a package
<code>go install golang.org/x/tools/cmd/goimports@latest</code>	Install a tool into <code>\$GOPATH/bin</code>	Once

DO THIS NOW • NO AI, JUST YOU

Run `go doc fmt.Printf`, then `go doc net/http.Server`, then `go doc -all strings | head -80`. Notice that the documentation is the source's comments, so what you see is exactly what `pkg.go.dev` shows. This is your primary reference from now on.

Anatomy of a program

```
// main.go
package main // an executable's package is always "main"

import (
    "fmt"      // standard library packages are imported by path
    "os"
    "strings"
)

func main() { // the entry point: no arguments, no return value
    args := os.Args[1:]
    if len(args) == 0 {
        fmt.Fprintln(os.Stderr, "usage: greet NAME...")
        os.Exit(2) // non-zero exit code signals failure to the shell
    }
    fmt.Printf("Hello, %s!\n", strings.Join(args, " and "))
}
```

Run it with `go run . Ada Grace`. Three things to notice: imports are grouped in parentheses and sorted by the formatter; `os.Args[0]` is the program name, as in C; unused imports would fail the build, so `goimports` manages them for you.

Project layout for the four projects

Go does not enforce a layout, and small programs should stay flat. Grow into structure only when a second package earns its place.

```
taskr/                                # Project 1: flat is fine
  go.mod
  main.go
  store.go
  store_test.go

shelf/                                # Project 3: a service with internal packages
  go.mod
  cmd/shelf/main.go                  # wiring: flags, config, start server
  internal/                          # "internal" is enforced: only this module can import it
    api/                            handlers.go, middleware.go, handlers_test.go
    store/                          store.go, sqlite.go, memory.go, store_test.go
  web/
    templates/ index.html
    static/    app.js, style.css
```

`cmd/<name>/main.go` for executables and `internal/` for private packages is the layout most Go teams use. Avoid `pkg/`, `utils/`, `models/`, `helpers/`; name packages after what they provide (`store`, `api`, `auth`), never after what they contain.

Cross-compilation for free

```
G00S=linux GOARCH=amd64 go build -o bin/taskr-linux .
G00S=windows GOARCH=amd64 go build -o bin/taskr.exe .
```

No toolchain to install, no container to spin up. This is why Go is the default language for CLI tools and infrastructure software.

CHAPTER 4

The AI-free method

You are not learning Go in a vacuum; you are learning it to prove to yourself that you can still build software with your own head. This chapter is the operating manual for that goal. It is short, but the rest of the book assumes you follow it.

Why the fear is legitimate, and why Go is the right antidote

Heavy use of code-generation tools erodes three specific skills: holding a program's structure in your head, reading error messages carefully, and tolerating the discomfort of not knowing. None of these are about typing speed; they are about attention. Go is a good language for rebuilding them because its compiler is fast and its error messages are unusually precise, so the feedback loop that trains attention is measured in seconds, not minutes.

The three zones

ZONE	ALLOWED	NOT ALLOWED
Reading (chapters)	Book, official docs, running the fragments	Asking a model to summarize or explain
Building (milestones)	Book, <code>go doc</code> , <code>pkg.go.dev</code> , Go Tour, Go by Example, your own experiments, the compiler	Any model, any autocomplete, any Stack Overflow <i>answer</i> copied without understanding
Reviewing (after "done")	Optional: a model as a reviewer of finished, committed code	Accepting generated replacements; using the review to "fix" a project that isn't finished

The line between the last two zones is a git commit tagged `done`. Until it exists, you are alone with the compiler.

How to get unstuck without a model

1. **Read the error message twice, out loud.** Go's messages name the file, line, column, the identifier and often the type. `cannot use s (variable of type string) as int value in argument to f` tells you everything. Appendix C decodes the thirty messages you will see most.
2. **Shrink the problem.** Copy the failing part into a scratch `main.go` in a `/tmp/x` module. Twenty lines you understand beat two hundred you don't.
3. **Print.** `fmt.Printf("%+v\n", value)` prints a struct with field names; `%T` prints its type; `%#v` prints Go syntax. Old-fashioned and unreasonably effective.
4. **Check the docs for the exact function.** `go doc strings.Fields`. Read the signature and the one-sentence description. Most confusion about a standard-library function dissolves on reading its signature.
5. **Write the test you wish passed.** A failing test with a concrete input and expected output turns a vague feeling of wrongness into a precise question.

6. **Use the debugger when prints aren't enough.** Delve (`dlv`) is integrated into VS Code; set a breakpoint, press F5. You will need it less than you think.
7. **Then**, after 25 minutes, search the web for the *concept*, not your specific code: “go slice append aliasing”, “go http handler json decode”. Read the first official or Go-blog result. Close the tab. Implement.

Retention habits that actually work

- **Start every session with a two-minute retrieval.** Before opening yesterday's code, write from memory the key fragment you learned (a worker pool, a JSON handler). Compare. The gap is your study list.
- **Explain it to the log.** In `LEARNING.md`, write one paragraph in your own words about the concept you used today. If you can't, you don't know it yet.
- **Rebuild once.** After finishing Project 1, delete it and rebuild it from the spec in 90 minutes. The second build is where the knowledge consolidates. Chapter 21 schedules this.
- **Read one standard-library file a week.** Go's standard library is written to be read. Start with `strings.Builder`, then `sync.Once`, then `net/http's ServeMux`. Use “go to definition” in your editor to jump into them.

What “understanding deeply” means here

You understand a Go concept deeply when you can: (1) write the idiom from memory, (2) explain why the language works that way, (3) predict what the code will do before you run it, and (4) name the gotcha. Every chapter's self-check questions test exactly these four. Every project's definition of done requires them. That is the bar; the book will hold you to it.

PART II

The core language

Six chapters that cover everything you need to write a real command-line program: syntax, data, errors, interfaces, a taste of generics, and tests. They culminate in Project 1, a task tracker you will actually use.

Chapter 5 · Syntax essentials

Chapter 6 · Data: structs, methods, pointers, slices, maps, strings

Chapter 7 · Errors as values

Chapter 8 · Interfaces

Chapter 9 · Generics, the 20% version

Chapter 10 · Testing

Project 1 · taskr, a task-tracking CLI

CHAPTER 5

Syntax essentials

Go's syntax fits on two pages. This chapter is those two pages, plus the details that distinguish someone who has read the Tour from someone who writes Go.

Variables, constants and zero values

```
var count int           // declared, zero value 0
var name = "shelf"      // type inferred: string
port := 8080            // short declaration; only inside functions
a, b := 1, "two"        // multiple, mixed types
x, y := 0, 0
x, y = y, x             // swap; = assigns, := declares

const maxRetries = 3    // untyped constant: adapts to context
const timeout time.Duration = 5 * time.Second
```

`:=` declares *and* assigns; it must introduce at least one new variable on its left side. Redeclaring in the same scope is a compile error; shadowing in an inner scope is legal and a classic source of bugs (see the gotcha below). Package-level variables must use `var`.

Every type has a zero value: `0`, `0.0`, `false`, `""`, and `nil` for pointers, slices, maps, channels, functions and interfaces. Go code is written to make zero values useful: an unset `sync.Mutex` is unlocked, an unset `bytes.Buffer` is empty and usable, an unset slice is `nil` and can be appended to.

GOTCHA

Shadowing with `:=`. Inside an `if` or `for`, `err := f()` creates a *new* `err` that dies at the closing brace, leaving the outer `err` untouched. If you meant to assign to the outer variable, use `=`. `go vet` does not catch this; your eyes must. The habit that prevents it: declare `var err error` only when you genuinely need it across blocks, and otherwise keep each `err` scoped to one call.

Enumerations with `iota`

Go has no `enum`. The convention is a named integer type, a `const` block, and `iota`, a counter that starts at 0 in each `const` block:

```
type Status int

const (
    StatusOpen Status = iota // 0
    StatusDone               // 1 (repeats the expression with iota=1)
    StatusArchived           // 2
)

func (s Status) String() string { // makes fmt print the name, not the number
    switch s {
    case StatusOpen:
        return "open"
    case StatusDone:
        return "done"
    case StatusArchived:
        return "archived"
    }
    return fmt.Sprintf("Status(%d)", int(s))
}
```

Basic types and conversions

`int` (platform word size, use it by default), `int64`, `uint8` (alias `byte`), `int32` (alias `rune`, a Unicode code point), `float64`, `bool`, `string`. There are **no implicit numeric conversions**: adding an `int` to an `int64` is a compile error; you write `int64(n)`. This feels pedantic for a day and then prevents a whole class of bugs. Division of integers truncates. Use `time.Duration` for durations, never bare integers of milliseconds.

Functions

```
// Multiple return values are the norm; the error is always last.
func parsePort(s string) (int, error) {
    n, err := strconv.Atoi(s)
    if err != nil {
        return 0, fmt.Errorf("parse port %q: %w", s, err)
    }
    if n < 1 || n > 65535 {
        return 0, fmt.Errorf("port %d out of range", n)
    }
    return n, nil
}

// Variadic parameters become a slice inside the function.
func sum(nums ...int) int {
    total := 0
    for _, n := range nums {
        total += n
    }
    return total
}
sum(1, 2, 3)
sum(mySlice...) // spread an existing slice

// Functions are values; closures capture variables by reference.
func counter() func() int {
    n := 0
    return func() int { n++; return n }
}
```

Go has no default parameter values, no keyword arguments and no overloading. When a function needs many optional settings, pass a struct (`http.Server{Addr: ":8080", Handler: mux}`) and let the zero values be the defaults. Named result parameters exist, but idiomatic code uses them rarely; the multiple-return form above is what you should write.

Control flow

if with an init statement scopes a variable to the condition and its blocks:

```
if v, ok := cache[key]; ok {
    return v
}
```

for is the only loop, in four shapes:

<code>for i := 0; i < 10; i++ { }</code>	<i>// classic</i>
<code>for cond { }</code>	<i>// while</i>
<code>for { }</code>	<i>// forever; exit with break or return</i>
<code>for i, v := range items { }</code>	<i>// index and copy of element (slice/array)</i>
<code>for k, v := range m { }</code>	<i>// key and value (map; random order)</i>
<code>for i, r := range "hélllo" { }</code>	<i>// byte index and rune (string)</i>
<code>for v := range ch { }</code>	<i>// until channel closed</i>
<code>for i := range 5 { }</code>	<i>// 0..4 (Go 1.22+)</i>
<code>for range 3 { }</code>	<i>// three times, no variable</i>

Use `_` to discard a value you don't need (for `_`, `v := range items`). `break` and `continue` work as expected; labelled versions exist for nested loops.

VERSION NOTE

Since Go 1.22, each iteration of a `for` loop gets a fresh copy of the loop variable. Before 1.22, closures launched inside a loop all saw the last value, one of Go's most famous footguns. If you read older tutorials that copy `v := v` inside a loop, that is why; you no longer need it.

`switch` does not fall through by default (the opposite of JavaScript), accepts multiple values per case, and can omit the expression entirely to act as a cleaner `if/else if` chain:

```
switch ext := filepath.Ext(name); ext {
case ".jpg", ".jpeg", ".png":
    return "image"
case ".md":
    return "markdown"
default:
    return "unknown"
}

switch {
case n < 0:
    return "negative"
case n == 0:
    return "zero"
default:
    return "positive"
}
```

`defer` schedules a call to run when the *enclosing function* returns, in last-in-first-out order. Its arguments are evaluated immediately, at the `defer` statement:

```
func process(path string) error {
    f, err := os.Open(path)
    if err != nil {
        return err
    }
    defer f.Close() // runs on every return path below, including panics

    mu.Lock()
    defer mu.Unlock()
    // ... work
    return nil
}
```

`defer` is Go's `finally`, `with`, and destructor in one keyword. Use it for closing files, unlocking mutexes, cancelling contexts, and finishing timers. Do not `defer` inside a long loop (the calls pile up until the function exits); extract the body into a function instead.

Packages and visibility

A directory is a package. All files in it start with the same `package name` line. An identifier is **exported** (usable from other packages) if and only if it starts with a capital letter: `Task`, `NewStore`, `ErrNotFound`. Lowercase identifiers are private to the package. There is no `public/private` keyword and no per-file visibility. Import a package by path and refer to its exported names through the package name: `store.NewStore()`, `json.Marshal()`.

The `fmt` verbs you actually use

VERB	MEANING	EXAMPLE OUTPUT
<code>%v</code>	Default format for any value	<code>{1 buy milk false}</code>
<code>%+v</code>	Struct with field names	<code>{ID:1 Title:buy milk Done:false}</code>
<code>%#v</code>	Go-syntax representation	<code>main.Task{ID:1, Title:"buy milk", Done:false}</code>
<code>%T</code>	Type of the value	<code>main.Task</code>
<code>%d %s %f %t</code>	Integer, string, float, bool	<code>42, hi, 3.140000, true</code>
<code>%q</code>	Quoted string	<code>"buy milk"</code>
<code>%w</code>	Wrap an error (only in <code>fmt.Errorf</code>)	see Chapter 7
<code>%5d %-10s %.2f</code>	Width, left-align, precision	<code>42, hi, 3.14</code>

`fmt.Println` adds spaces and a newline; `fmt.Printf` needs an explicit `\n`; `fmt.Sprintf` returns a string; `fmt.Fprintf(w, ...)` writes to any `io.Writer`, including files and HTTP responses.

Self-check

1. Why is `x := 1` illegal at package level, and what do you write instead?
2. What does `defer fmt.Println(i)` print if `i` changes after the `defer` line?
3. Write, from memory, a `switch` with no expression that classifies an HTTP status code as success, client error or server error.

CHAPTER 6

Data: structs, methods, pointers, slices, maps, strings

Go has one way to model records (structs), two workhorse collections (slices and maps), and one deceptively simple string type. Almost every bug a Python or TypeScript developer writes in their first month of Go comes from the memory model of one of these four.

Structs

```
type Task struct {
    ID      int      `json:"id"`
    Title   string   `json:"title"`
    Done    bool     `json:"done"`
    CreatedAt time.Time `json:"created_at"`
    Tags    []string `json:"tags,omitempty" // omit from JSON when empty`
}

t := Task{ID: 1, Title: "read chapter 6"} // keyed literal; unset fields are zero
p := &Task{Title: "pointer to a task"}   // pointer to a fresh struct
var empty Task                           // all zero values, ready to use
t.Done = true
```

Always use keyed literals (`Field: value`); positional literals break when a field is added. The backtick strings after fields are **struct tags**, metadata read via reflection by packages like `encoding/json` to map field names. Two struct values are comparable with `==` if all their fields are comparable, which makes small structs usable as map keys.

Methods and receivers

A method is a function with a *receiver* declared between `func` and the name. The receiver can be a value or a pointer:

```
func (t Task) Summary() string { // value receiver: gets a copy
    status := " "
    if t.Done {
        status = "x"
    }
    return fmt.Sprintf("[%s] %d %s", status, t.ID, t.Title)
}

func (t *Task) MarkDone() { // pointer receiver: mutates the caller's Task
    t.Done = true
}

t := Task{ID: 1, Title: "x"}
t.MarkDone() // Go takes &t automatically for addressable values
fmt.Println(t.Summary())
```

RULE OF THUMB

Use a pointer receiver when the method mutates the receiver, when the struct is large, or when *any* method on the type already needs a pointer (be consistent across a type). Use a value receiver for small, immutable value types like `time.Duration` or your `Status` enum. When in doubt, pointer.

The set of methods a type has matters for interfaces (Chapter 8): `*Task` has both `Summary` and `MarkDone`; a plain `Task` has only `Summary`. If a function expects an interface with `MarkDone`, you must pass `&t`, not `t`.

Constructors are ordinary functions named `NewX` that return `*X` (or `X` for small values), and they exist only when the zero value isn't usable or validation is needed:

```
func NewStore(path string) (*Store, error) {
    if path == "" {
        return nil, errors.New("store: empty path")
    }
    return &Store{path: path, tasks: map[int]Task{}}, nil
}
```

Embedding: composition instead of inheritance

Embedding a type in a struct promotes the embedded type's fields and methods to the outer type:

```
type Logger struct{ prefix string }
func (l Logger) Logf(format string, args ...any) {
    fmt.Printf(l.prefix+format+"\n", args...)
}

type Server struct {
    Logger           // embedded: Server now has .Logf and .prefix
    addr string
}

s := Server{Logger: Logger{prefix: "[srv] "}, addr: ":8080"}
s.Logf("listening on %s", s.addr) // promoted method
```

This is not inheritance: `Server` is not a `Logger`, there is no virtual dispatch, and `Logger`'s methods cannot see `Server`'s fields. It is plain composition with syntactic sugar. The most common real use is embedding an interface or a `sync.Mutex` in a struct so the outer type gets its methods.

Pointers

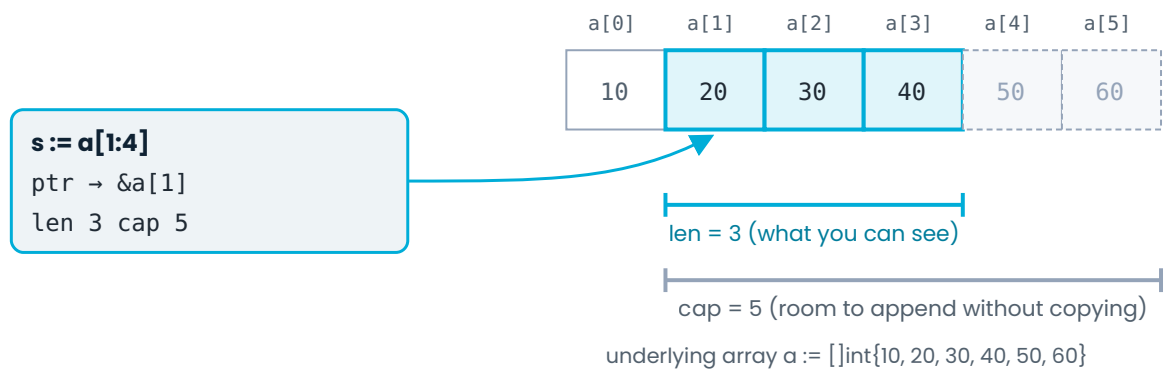
`&x` takes the address of `x`; `*p` dereferences `p`; `new(T)` allocates a zeroed `T` and returns `*T` (usually written `&T{}` instead). There is no pointer arithmetic. A `nil` pointer dereference panics; check before you use anything that might be `nil`.

When do you use a pointer? To let a function mutate its argument, to avoid copying a large struct, to express "optional" (`*time.Time` is either a `time` or `nil`), and always for receivers on types you mutate. Basic values, small structs and everything that is conceptually a *value* (points, money

amounts, durations) are passed by value. Do not micro-optimize this: the compiler's escape analysis puts values on the stack when it can, and a small struct copy is cheaper than the indirection.

Slices: the one data structure you must understand deeply

An **array** (`[4]int`) has a fixed length that is part of its type; you will rarely use one directly. A **slice** (`[]int`) is a small header, three words long, that *views* a contiguous region of an underlying array:



A slice is (pointer, length, capacity). Two slices can share one array; appending within capacity writes into that shared array.

Everything surprising about slices follows from that picture.

```
var s []int           // nil slice: len 0, cap 0, safe to range and append
s = append(s, 1, 2, 3) // append may allocate a new array and returns the new header
t := s[1:]           // t shares s's array: t[0] is s[1]
t[0] = 99             // s is now [1 99 3]
u := make([]int, 0, 10) // len 0, cap 10: preallocate when you know the size
len(s), cap(s)         // 3, and whatever growth policy gave (often 3 or 4 here)
```

GOTCHA

Aliasing through `append`. If `a` has spare capacity, `b := append(a[:2], 100)` writes `100` into `a`'s array, silently changing `a[2]`. If it has no spare capacity, `append` copies to a new array and `a` is untouched. Whether you clobber the original depends on capacity, which is why this bug appears intermittently. Rules: always assign the result of `append` back (`s = append(s, x)`); when you hand a sub-slice to code that may append, give it a copy (`slices.Clone(s)`) or cap it with a full slice expression `s[low:high:high]` so an append is forced to reallocate.

Idioms you will use daily:

```
s = append(s, other...)           // concatenate
s = slices.Delete(s, i, i+1)      // remove element i (shifts left, reuses array)
s = slices.Insert(s, i, v)        // insert at i
slices.Sort(s)                   // in place, for ordered element types
slices.SortFunc(tasks, func(a, b Task) int {
    return cmp.Compare(a.CreatedAt.Unix(), b.CreatedAt.Unix())
})
idx := slices.IndexFunc(tasks, func(t Task) bool { return t.ID == id }) // -1 if absent
if slices.Contains(tags, "urgent") { }
c := slices.Clone(s)              // independent copy
n := copy(dst, src)              // copies min(len(dst), len(src)) elements
```

`range` over a slice yields a **copy** of each element. `for _, t := range tasks { t.Done = true }` modifies the copies and discards them. To mutate in place, index: `for i := range tasks { tasks[i].Done = true }`, or store pointers.

A `nil` slice and an empty non-`nil` slice behave identically for `len`, `range` and `append`; the difference only shows up in JSON (`null` vs `[]`) and in `== nil` checks. Return `nil` for “nothing” unless the JSON shape matters.

Maps

```
m := map[string]int{}           // or make(map[string]int); a nil map panics on write
m["a"] = 1
v := m["missing"]               // zero value (0), no error
v, ok := m["missing"]           // ok == false distinguishes "absent" from "zero"
delete(m, "a")                  // no-op if absent
for k, v := range m {           // iteration order is deliberately randomized
    len(m)

    // deterministic order: sort the keys (Go 1.23+; maps.Keys returns an iterator)
    for _, k := range slices.Sorted(maps.Keys(m)) {
        fmt.Println(k, m[k])
    }
}
```

Keys must be comparable (strings, numbers, booleans, pointers, structs of comparable fields; not slices, maps or functions). Maps are reference-like: passing one to a function lets the function mutate the caller’s map. Maps are **not safe for concurrent writes**; guard them with a `sync.Mutex` (Chapter 13). If the map holds struct values, `m[k].Done = true` is a compile error because the value isn’t addressable; read, modify, write back (`t := m[k]; t.Done = true; m[k] = t`) or store `*Task`.

Strings, bytes and runes

A Go string is an immutable sequence of **bytes**, conventionally UTF-8. This has consequences:

```
s := "héllö"
len(s)           // 6 bytes, not 5 characters: é is two bytes
s[1]             // a byte (uint8), 0xC3, not 'é'
for i, r := range s { } // i is the byte offset, r is the rune (code point)
utf8.RuneCountInString(s) // 5
[]rune(s)        // slice of code points; index it for "characters"
[]byte(s)        // mutable copy of the bytes
string(b)        // back to string (copies)
```

For building strings in a loop use `strings.Builder`, not `+=`:

```
var sb strings.Builder
for _, t := range tasks {
    fmt.Fprintf(&sb, "%d\t%s\n", t.ID, t.Title)
}
out := sb.String()
```

Functions from `strings` you will reach for: `Split`, `Fields` (split on whitespace runs), `Join`, `TrimSpace`, `HasPrefix`, `Contains`, `ToLower`, `ReplaceAll`, `Cut` (split once, returns before/after/found). From `strconv`: `Atoi`, `Itoa`, `ParseInt`, `ParseBool`, `Quote`. From `unicode`: `IsLetter`, `IsSpace`. Never convert to `[]rune` “just in case”; only do it when you truly need character indexing.

Self-check

1. Draw the slice header for `s := make([]int, 2, 5)` and then for `t := s[1:3]`. What is `cap(t)`?
2. Why does `for _, t := range tasks { t.Done = true }` do nothing useful, and what are the two fixes?
3. What happens on `var m map[string]int; m["a"] = 1`? How about reading `m["a"]` from the same nil map?
4. Predict the output of `fmt.Println(len("日本"))` and `fmt.Println(utf8.RuneCountInString("日本"))`.

CHAPTER 7

Errors as values

Error handling is the part of Go that newcomers complain about and veterans defend most fiercely. Learn the four idioms in this chapter and the complaints go away, because every function's failure modes become visible at every call site.

The basic contract

`error` is an interface with one method: `Error() string`. Any type that has that method is an error. The conventions are simple and universal:

- A function that can fail returns `error` as its **last** result.
- On success the error is `nil`; on failure the other results are unspecified (usually zero values) and must not be used.
- The caller checks `if err != nil` **immediately**, and either handles the error or returns it upward with context added.

```
data, err := os.ReadFile(path)
if err != nil {
    return nil, fmt.Errorf("load tasks from %s: %w", path, err)
}
```

Handle or return, never both. Do not log an error and also return it; the caller will log it too and you'll get every failure three times. The place errors get logged is the top of the program (`main`, or an HTTP middleware).

Wrapping with `%w`

`fmt.Errorf` with the `%w` verb wraps an error inside a new one, preserving the original for inspection while adding context. The message reads like a path from the outside in:

```
load tasks from tasks.json: open tasks.json: no such file or directory
```

Add context that the caller can't already see: which file, which task ID, which operation. Don't prefix with "error:" or "failed to" (the caller knows it failed), don't capitalize, and don't end with punctuation, so the messages chain cleanly.

Sentinel errors and `errors.Is`

A **sentinel** is a package-level error variable that callers compare against:

```
var ErrNotFound = errors.New("task not found")

func (s *Store) Get(id int) (Task, error) {
    t, ok := s.tasks[id]
    if !ok {
        return Task{}, fmt.Errorf("get task %d: %w", id, ErrNotFound)
    }
    return t, nil
}

// caller
t, err := store.Get(42)
if errors.Is(err, ErrNotFound) { // true even though err is wrapped
    fmt.Println("no such task")
} else if err != nil {
    return err
}
```

`errors.Is` walks the wrap chain, so callers can react to a specific condition regardless of how many layers added context. Never compare with `==` or by string matching. The standard library is full of sentinels: `io.EOF`, `os.ErrNotExist`, `sql.ErrNoRows`, `context.Canceled`.

Custom error types and `errors.As`

When the caller needs *data* about the failure, define a type:

```
type ValidationError struct {
    Field string
    Msg    string
}

func (e *ValidationError) Error() string {
    return fmt.Sprintf("invalid %s: %s", e.Field, e.Msg)
}

func validate(t Task) error {
    if strings.TrimSpace(t.Title) == "" {
        return &ValidationError{Field: "title", Msg: "must not be empty"}
    }
    return nil
}

// caller: pull the typed error out of the chain
var ve *ValidationError
if errors.As(err, &ve) {
    http.Error(w, ve.Msg, http.StatusBadRequest)
    return
}
```

Use pointer receivers for error types and return pointers, so equality is by identity and `errors.As` has a concrete target. Keep the number of custom error types small; two or three per project is typical.

panic and recover

`panic` unwinds the stack running deferred calls, and crashes the program with a stack trace unless a deferred `recover()` catches it. Use it only for programmer errors that make continuing meaningless: an impossible state, a `nil` that “can’t” be `nil`, a `Must`-style helper failing at start-up (`template.Must`, `regexp.MustCompile`). Never use it for expected failures such as bad user input, missing files or network errors.

The one place you write `recover` yourself is an HTTP middleware that turns a panicking handler into a 500 response instead of killing the whole server (Chapter 17).

Errors in main

`main` cannot return an error, so the idiom is a `run` function that can:

```
func main() {
    if err := run(os.Args[1:], os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, "taskr:", err)
        os.Exit(1)
    }
}

func run(args []string, out io.Writer) error {
    // all real logic here; testable, returns errors
}
```

This makes the whole program testable (`run` takes its inputs and output as parameters) and gives you a single place where errors are printed and exit codes decided. `log.Fatal` is acceptable in `main` for start-up failures but nowhere else, because it calls `os.Exit` and skips deferred functions.

Combining errors

`errors.Join(err1, err2)` (Go 1.20+) bundles several errors into one that satisfies `errors.Is` for each; useful when validating many fields or closing many resources. `%w` may appear more than once in one `fmt.Errorf` for the same effect.

Self-check

1. Why must you not use `==` to test for `ErrNotFound` once the error has been wrapped?
2. Write a function `load(path string) ([]Task, error)` that wraps both the file error and the JSON error with context. How many `if err != nil` blocks does it have?
3. Under what circumstances is `panic` acceptable? Name two functions in the standard library that panic by design.

CHAPTER 8

Interfaces

Interfaces are how Go achieves polymorphism, decoupling and testability without inheritance. They are also the reason a Go program can print to a terminal, a file, a network socket and an in-memory buffer with the same line of code.

Implicit satisfaction

An interface is a set of method signatures. A type satisfies an interface by having those methods; there is no `implements` keyword and the type's author does not need to know the interface exists:

```
type Store interface {
    Add(t Task) (Task, error)
    List() ([]Task, error)
    Complete(id int) error
}

type FileStore struct{ path string } // in one package...
func (s *FileStore) Add(t Task) (Task, error) { /* ... */ }
func (s *FileStore) List() ([]Task, error) { /* ... */ }
func (s *FileStore) Complete(id int) error { /* ... */ }

type MemStore struct{ tasks map[int]Task } // ...and another implementation for tests
// same three methods

var _ Store = (*FileStore)(nil) // compile-time assertion: *FileStore satisfies Store
var _ Store = (*MemStore)(nil)
```

The blank-identifier assertion costs nothing at runtime and turns “I forgot a method” into a compile error at the right place. Use it for every implementation you care about.

MENTAL SHIFT · COMING FROM PYTHON / TYPESCRIPT

TypeScript interfaces are also structural, so this will feel familiar. The differences: Go interfaces contain only methods (no fields), they are checked at compile time when you assign a concrete value to an interface variable, and they are typically tiny. A Go interface with more than four methods is a smell; the most used interfaces in the standard library have one.

The interfaces that run the world: `io.Reader` and `io.Writer`

```
type Reader interface { Read(p []byte) (n int, err error) }
type Writer interface { Write(p []byte) (n int, err error) }
```

Files, network connections, HTTP request bodies, HTTP response writers, `bytes.Buffer`, `strings.Builder`, gzip streams, and hashers all implement one or both. That is why these lines all work:

```
fmt.Fprintf(os.Stdout, "hello %s\n", name)    // terminal
fmt.Fprintf(w, "hello %s\n", name)           // http.ResponseWriter
fmt.Fprintf(&buf, "hello %s\n", name)        // *bytes.Buffer, in tests
json.NewDecoder(r.Body).Decode(&req)         // r.Body is an io.Reader
json.NewEncoder(f).Encode(tasks)             // f is an *os.File
io.Copy(dst, src)                           // any Writer <- any Reader
```

When you write a function that produces output, take an `io.Writer` parameter instead of printing. When you consume input, take an `io.Reader`. Your code becomes testable and composable for free; this single habit distinguishes idiomatic Go.

Other small interfaces you meet constantly

`fmt.Stringer` (`String() string`) controls how `fmt` prints your type. `error` is an interface. `sort.Interface` is the older way to sort (prefer `slices.SortFunc`). `http.Handler` (`ServeHTTP(w, r)`) is the heart of `net/http`. `io.Closer`, `io.ReadCloser` and friends are compositions: Go builds bigger interfaces by embedding smaller ones.

any, type assertions and type switches

`any` (alias for `interface{}`) is satisfied by every type. You will see it in `json.Unmarshal(data, &v)` where `v` is `any`, in `fmt.Println(args ...any)`, and in generic constraints. To get the concrete value back out you assert:

```
v, ok := x.(string)    // ok is false if x isn't a string; without ok, a failure panics

switch v := x.(type) { // type switch: v has the concrete type inside each case
case string:
    return strings.ToUpper(v)
case int, int64:
    return fmt.Sprint(v)
case fmt.Stringer:
    return v.String()
case nil:
    return "<nil>"
default:
    return fmt.Sprintf("%v", v)
}
```

Reaching for `any` in your own APIs is almost always wrong; use a concrete type, a small interface or a generic type parameter instead.

Design rules

RULE OF THUMB

Accept interfaces, return structs. Functions take the narrowest interface that describes what they need (`io.Reader`, not `*os.File`) and return concrete types so callers get the full API. **Define interfaces where they are used**, not where they are implemented: the `api` package that needs a store declares `type Store interface{...}` with exactly the methods it calls, and the `store` package knows nothing about it. **Don't design interfaces up front.** Write the concrete type first; extract an interface when the second implementation (usually a test fake) appears.

Two gotchas that cause real production bugs

GOTCHA

A nil pointer inside an interface is not a nil interface. If `var p MyErr = nil` and you return `p` as an error, the returned interface holds (type `MyErr`, value `nil`) and `err != nil` is **true**. Always return the literal `nil` for the no-error case, never a typed nil pointer.

GOTCHA

Method sets and pointer receivers. If `Complete` has a pointer receiver, only `*FileStore` satisfies `Store`; a `FileStore` value does not, and the compiler error says so in a way that confuses everyone once: “FileStore does not implement Store (method Complete has pointer receiver)”. Fix: pass `&fs`, or construct with `&FileStore{}`.

Self-check

1. Write a function `render(w io.Writer, tasks []Task) error` and then call it with `os.Stdout` and with a `*bytes.Buffer` in a test. Why doesn't it need two versions?
2. What does `var _ Store = (*MemStore)(nil)` do, and what does it cost at runtime?
3. Explain in one sentence why a `Store` interface belongs in the `api` package rather than the `store` package.

CHAPTER 9

Generics, the 20% version

Go got type parameters in 1.18 after a decade of debate. They are deliberately less powerful than TypeScript's, and idiomatic Go uses them sparingly: for containers and utility functions, almost never for business logic.

Type parameters on functions

```
// Map applies f to every element and returns the results.
func Map[T, U any](xs []T, f func(T) U) []U {
    out := make([]U, 0, len(xs))
    for _, x := range xs {
        out = append(out, f(x))
    }
    return out
}

titles := Map(tasks, func(t Task) string { return t.Title }) // T and U inferred

func Filter[T any](xs []T, keep func(T) bool) []T {
    var out []T
    for _, x := range xs {
        if keep(x) {
            out = append(out, x)
        }
    }
    return out
}
```

Type arguments are usually inferred from the arguments; you only write `Map[Task, string](...)` when inference fails.

Constraints

`any` allows every type but permits no operations except assignment. To compare, add or order, you constrain:

```
func Max[T cmp.Ordered](a, b T) T { // cmp.Ordered: ints, floats, strings
    if a > b {
        return a
    }
    return b
}

func Uniq[T comparable](xs []T) []T { // comparable: anything usable as a map key
    seen := make(map[T]struct{})
    var out []T
    for _, x := range xs {
        if _, ok := seen[x]; !ok {
            seen[x] = struct{}{}
            out = append(out, x)
        }
    }
    return out
}

type Number interface { ~int | ~int64 | ~float64 } // ~ = "underlying type": type Celsius
float64 qualifies
func Sum[T Number](xs []T) T {
    var total T
    for _, x := range xs {
        total += x
    }
    return total
}
```

Generic types

```
type Stack[T any] struct{ items []T }

func (s *Stack[T]) Push(v T) { s.items = append(s.items, v) }
func (s *Stack[T]) Pop() (T, bool) {
    var zero T
    if len(s.items) == 0 {
        return zero, false
    }
    v := s.items[len(s.items)-1]
    s.items = s.items[:len(s.items)-1]
    return v, true
}

var st Stack[int]
```

VERSION NOTE

Go 1.27 (August 2026) added *generic methods*: a method may declare its own type parameters, so you no longer need a package-level function for “a generic operation in the namespace of a type”. You will see this in newer code; you do not need it for the projects in this book.

When not to use generics

Before Go 1.18, `Map`/`Filter` did not exist and Go programmers wrote loops. Most still do, because a five-line `for` loop is clearer than a chain of generic helpers and the compiler produces better code for it. Use generics when you would otherwise copy-paste a function for several types (a cache, a set, a min-heap, `Max`). Do not use them to make code “more abstract”; Go culture will read that as a smell. The `slices` and `maps` packages already cover most generic needs.

Self-check

1. Why does `func Sum[T any](xs []T) T` fail to compile?
2. What is the difference between the constraints `any`, `comparable` and `cmp.Ordered`?

CHAPTER 10

Testing

Go programmers debug by writing tests. The testing tool is in the standard library, needs no configuration, and runs in milliseconds. This chapter gives you the one pattern that covers 80% of tests you'll ever write.

The mechanics

A test lives in a file ending in `_test.go` in the same directory as the code. It is a function named `TestXxx` taking `*testing.T`. `go test` compiles these files with the package and runs them; they are excluded from normal builds.

```
// store_test.go
package main

import "testing"

func TestAddAssignsIncrementingIDs(t *testing.T) {
    s := NewMemStore()
    a, _ := s.Add(Task{Title: "first"})
    b, _ := s.Add(Task{Title: "second"})
    if a.ID != 1 || b.ID != 2 {
        t.Fatalf("got IDs %d and %d, want 1 and 2", a.ID, b.ID)
    }
}
```

`t.Errorf` records a failure and continues; `t.Fatalf` records and stops this test. Use `Fatal` when continuing would be meaningless (a nil you're about to dereference), `Errorf` otherwise. There are no assertion libraries in the standard library, on purpose: `if got != want` is clear and the failure message is whatever you decide is useful. Always print both `got` and `want`.

Table-driven tests with subtests

This is the pattern. Learn it once; it covers most units:

```
func TestParseCommand(t *testing.T) {
    tests := []struct {
        name    string
        args    []string
        want    Command
        wantErr bool
    }{
        {
            name: "add with title",
            args: []string{"add", "buy milk"},
            want: Command{Kind: "add", Title: "buy milk"},
        },
        {name: "done with id", args: []string{"done", "3"}, want: Command{Kind: "done",
ID: 3}},
        {name: "done with bad id", args: []string{"done", "x"}, wantErr: true},
        {name: "no args", args: nil, wantErr: true},
    }
    for _, tc := range tests {
        t.Run(tc.name, func(t *testing.T) {
            got, err := ParseCommand(tc.args)
            if (err != nil) != tc.wantErr {
                t.Fatalf("err = %v, wantErr = %v", err, tc.wantErr)
            }
            if !tc.wantErr && got != tc.want {
                t.Errorf("got %+v, want %+v", got, tc.want)
            }
        })
    }
}
```

Each row is a case with a name; `t.Run` makes it a subtest you can run alone: `go test -run 'TestParseCommand/bad_id' -v`. Adding a case is adding a line. When a bug is reported, you add the failing row first, then fix.

Useful helpers

- `t.TempDir()` gives a directory that is deleted after the test. Perfect for file-based stores.
- `t.Helper()` inside a helper function makes failures report the caller's line, not the helper's.
- `t.Cleanup(func())` registers teardown.
- `t.Parallel()` lets independent tests run concurrently, which also shakes out races.
- `testdata/` is a directory name the `go` tool ignores when building; put fixtures there.
- Package `store_test` (external test package, same directory) tests only the exported API and avoids import cycles.

Flags worth memorizing

```
go test ./...           # everything
go test -run TestParse -v ./... # filter by regex, verbose output
go test -race ./...      # data race detector; run it on all concurrent code
go test -cover ./...     # coverage summary
go test -count=1 ./...   # disable the test cache
go test -bench . -benchmem # run benchmarks
```

Benchmarks are functions named `BenchmarkXxx(b *testing.B)` whose body loops with `for b.Loop() { ... }` (Go 1.24+; older code uses `for i := 0; i < b.N; i++`). You will rarely need them in this book; know they exist.

What to test in the projects

Test behaviour through the public surface: the `run` function of a CLI with a fake stdout, the `Store` interface with both implementations, HTTP handlers with `httptest` (Chapter 17). Don't test private helpers directly unless they hold real complexity. A good target for these projects is a test per milestone acceptance criterion, not a coverage number.

Self-check

1. When would you choose `t.Fatalf` over `t.Errorf`?
2. Convert one of your Project 1 tests into a table-driven test with three rows from memory.
3. What does `go test -race` detect, and why should it run on every concurrent package?

PROJECT 1

taskr, a task-tracking CLI

taskr • a command-line task tracker with JSON persistence

Time box: 6–8 hours over 3–4 sessions · **Chapters needed:** 3, 5, 6, 7, 8, 10 · **Dependencies:** none (standard library only)

You will exercise: modules, structs and methods, slices and maps, errors and wrapping, interfaces, `encoding/json`, `os`, `flag`, `time`, sorting, table-driven tests

Why this project

A task tracker is small, has real state, and forces the core loop of every Go program: parse input, validate, mutate data, persist, report. Unlike FizzBuzz, it has a file format, error cases a user will actually hit, and a design decision (where does the storage interface live?) that pays off in Project 3. You should end up with something you use for a week.

What it looks like when finished

```
$ taskr add "Read chapter 7 on errors" --tag go --tag reading
Added task 1: Read chapter 7 on errors

$ taskr add "Renew passport"
Added task 2: Renew passport

$ taskr list
ID  STATUS  AGE   TITLE                                TAGS
1   open    2m    Read chapter 7 on errors             go, reading
2   open    10s   Renew passport

$ taskr done 1
Completed task 1

$ taskr list --all
ID  STATUS  AGE   TITLE                                TAGS
1   done    3m    Read chapter 7 on errors             go, reading
2   open    1m    Renew passport

$ taskr done 9
taskr: complete task 9: task not found
$ echo $?
1

$ taskr rm 2
Removed task 2
```

Tasks live in a JSON file (default `~/.taskr.json`, overridable with `--file` or the `TASKR_FILE` environment variable), so the state survives between runs and you can inspect it with any text editor.

Suggested layout

Flat, one package, four or five files:

```
taskr/
  go.mod
  main.go      // main() -> run(); flag parsing; dispatch to subcommands
  task.go      // Task struct, Status type, validation
  store.go     // Store interface, FileStore (JSON), MemStore (for tests)
  render.go    // table output to an io.Writer
  store_test.go
  main_test.go
  LEARNING.md
```

Milestones

M1 Model and in-memory store (1.5 h)

Define `Task` (ID, Title, Done, CreatedAt, CompletedAt as `*time.Time`, Tags) and a `Status` with a `String()` method. Write a `Store` interface with `Add`, `List`, `Get`, `Complete`, `Remove`. Implement `MemStore` backed by a `map[int]Task` with an incrementing ID counter. Return a wrapped `ErrNotFound` sentinel for missing IDs.

Accept when: `go test` passes tests for `Add` (IDs increment from 1), `Get` (found and not found via `errors.Is`), `Complete` (sets `Done` and `CompletedAt`), `Remove` (subsequent `Get` fails). `List` must return tasks sorted by ID regardless of map order.

M2 JSON persistence (1.5 h)

Implement `FileStore` that loads the whole file into memory on construction (a missing file means an empty store, not an error: use `errors.Is(err, os.ErrNotExist)`), and writes the whole file after every mutation. Write to a temporary file in the same directory and `os.Rename` it over the original so a crash mid-write never corrupts the store. Use struct tags to produce `snake_case` JSON and `omitempty` where appropriate.

Accept when: the same test suite from M1 passes against `FileStore` using `t.TempDir()`; a hand-edited JSON file loads correctly; a malformed file produces an error that names the file. Both stores pass the same tests: write the suite once as a function taking a `Store` and call it from two `Test` functions.

M3 Command-line interface (2 h)

In `main.go`, implement `run(args []string, stdout io.Writer) error`. Dispatch on `args[0]` (`add`, `list`, `done`, `rm`) and parse per-subcommand flags with a fresh `flag.NewFlagSet` for each. Print usage to `stderr` and return an error on unknown commands. Resolve the store path: `--file` flag, then `TASKR_FILE` env var, then `os.UserHomeDir() + /.taskr.json`.

Accept when: every command in the transcript above works end to end; unknown commands and bad IDs exit with status 1 and a one-line message prefixed `taskr: ;` `run` is tested with a `MemStore` and a `bytes.Buffer` for at least `add` and `done` (you'll need a way to inject the store; a package-level `newStore` function variable you override in tests is acceptable here).

M4 Rendering and polish (1 h)

Write `renderTable(w io.Writer, tasks []Task)` using `text/tabwriter` for aligned columns. Show a human "age" (`10s`, `2m`, `3h`, `2d`) computed from `time.Since`. `list` shows open tasks by default and everything with `--all`. Add `--tag` (repeatable: implement `flag.Value` on a `[]string` type) to `add`, and `--tag` filtering to `list`.

Accept when: the output matches the transcript in spirit; `go vet ./...` is clean; `gofmt -l .` prints nothing; `go build -o bin/taskr .` produces a binary you install on your `PATH` and use for real.

Definition of done

- ☐ All four milestones' acceptance criteria are met and the tests pass with `go test -count=1 ./...`.
- ☐ Both store implementations pass the same shared test suite.
- ☐ Every error returned from a store or file operation is wrapped with context, and `main` is the only place that prints errors.
- ☐ You can explain, without notes, why `Store` is an interface, why `FileStore` uses pointer receivers, and what happens to the JSON file if the program is killed during a write.
- ☐ The repository has at least six commits with messages that describe intent, and `LEARNING.md` has one entry per session.
- ☐ You have tagged the commit `done`. Only now may you ask an AI to review it, if you choose to.

Stretch goals (optional, pick one)

- `taskr edit ID --title "..."` with a validation error type surfaced as a friendly message.
- `taskr list --sort age|title` using `slices.SortFunc`.
- A `--json` flag on `list` that emits machine-readable output, so `taskr list --json | jq` works.
- Undo: keep the previous file as `.taskr.json.bak` and add `taskr undo`.

Rebuild checkpoint

One week after finishing, delete the directory and rebuild M1–M3 from this spec in a single 90-minute session. Do not look at your original. The parts you cannot reproduce are the parts to re-read. This is scheduled in Chapter 21.

PART III

Concurrency

Go's standout capability. Six short chapters build a precise mental model of goroutines, channels, mutexes and cancellation, then Project 2 makes you use all of them to build a concurrent link checker that runs under the race detector.

Chapter 11 · Goroutines and the scheduler

Chapter 12 · Channels and `select`

Chapter 13 · Sharing memory safely: `sync` and `atomic`

Chapter 14 · Cancellation with context

Chapter 15 · The five patterns that cover 80% of concurrent code

Chapter 16 · Pitfalls and the race detector

Project 2 · `linkcheck`, a concurrent link checker

CHAPTER 11

Goroutines and the scheduler

A goroutine is a function running concurrently with other functions, managed by the Go runtime rather than the operating system. Starting one costs a few kilobytes and a few hundred nanoseconds. That price changes how you design programs.

The `go` keyword

```
go fetch(url)           // run fetch(url) concurrently; returns immediately
go func() {             // anonymous function, the most common form
    process(item)
}()
```

Two facts define everything else. First, **when `main` returns, the program exits**, whether or not other goroutines are still running. Second, **there is no way to stop a goroutine from the outside**; it must notice a signal (a closed channel or a cancelled context) and return on its own. Concurrency in Go is therefore about two questions: how do I wait for goroutines to finish, and how do I tell them to stop?

Waiting with `sync.WaitGroup`

```
var wg sync.WaitGroup
for _, url := range urls {
    wg.Add(1)           // register before starting, in the parent goroutine
    go func() {
        defer wg.Done() // signal completion on every exit path
        check(url)
    }()
}
wg.Wait()              // block until the counter returns to zero
```

`Add` must happen before the `go` statement, in the goroutine that will `Wait`; calling it inside the child races with `Wait`. Go 1.25 added `wg.Go(func())` which does the `Add/Done` bookkeeping for you, and you will see it in new code.

MENTAL SHIFT · COMING FROM PYTHON / TYPESCRIPT

Coming from JavaScript, the closest analogy is `Promise.all` over a set of async calls, except that inside each goroutine you write plain blocking code: no `await`, no callbacks, no coloring of functions into sync and async. Coming from Python, the closest analogy is threads, except that they are cheap enough to make one per request or per URL, and the GIL does not exist: goroutines run truly in parallel across cores.

What the runtime does for you

The scheduler multiplexes goroutines (G) onto a small pool of OS threads (M), one per logical processor (P, controlled by `GOMAXPROCS`, which defaults to your core count). When a goroutine blocks on I/O, a channel, a mutex or `time.Sleep`, the scheduler parks it and runs another on the same thread. Goroutine stacks start at a few kilobytes and grow as needed. The practical consequences:

- **Blocking is free and normal.** Write straightforward sequential code inside each goroutine. Don't build callback chains.
- **Ten thousand goroutines is fine.** A million is fine if they're mostly waiting. What you limit is not goroutines but the *external* resource they hit (connections, file handles, a rate-limited API), using a semaphore or worker pool (Chapter 15).
- **Parallelism is automatic.** CPU-bound goroutines spread across cores without configuration.
- **You still need synchronization.** Two goroutines touching the same variable without coordination is a data race, and races are undefined behaviour in Go, not just "sometimes wrong".

Returning results from goroutines

A goroutine can't return a value to its starter. The three ways to get results out: send them on a channel (Chapter 12), write them into a shared structure under a mutex (Chapter 13), or write into a pre-allocated slice slot that only this goroutine owns:

```
results := make([]Result, len(urls))
var wg sync.WaitGroup
for i, url := range urls {
    wg.Add(1)
    go func() {
        defer wg.Done()
        results[i] = check(url)    // each goroutine writes a distinct index: no race
    }()
}
wg.Wait()
```

The last form is underrated: it is race-free because each index has exactly one writer, it preserves order, and it needs no channel. Use it whenever the number of tasks is known up front.

Self-check

1. What happens if `main` starts ten goroutines and immediately returns?
2. Why must `wg.Add(1)` be called before the `go` statement rather than inside the goroutine?
3. Is writing `results[i]` from many goroutines a data race? Why or why not?

CHAPTER 12

Channels and select

A channel is a typed conduit through which goroutines send and receive values. It combines a queue with synchronization: a send and a receive are a handshake, and that handshake is what makes channels safe without locks.

Creating, sending, receiving

```
ch := make(chan int)           // unbuffered: capacity 0
buf := make(chan string, 8)    // buffered: capacity 8

ch <- 42                       // send (blocks until someone receives, if unbuffered)
v := <-ch                      // receive (blocks until someone sends)
v, ok := <-ch                 // ok is false once the channel is closed and drained
close(ch)                     // only the sender closes; receives then yield zero,
ok=false
for v := range ch { }         // receive until closed
```

Unbuffered: a rendezvous sender receiver Both block until the other arrives. The send completes exactly when the receive does: a synchronization point, not a queue.

Buffered (cap 3): a small queue sender 7 3 receiver Sends succeed immediately while there is room; the sender blocks only when the buffer is full. Receivers block only when it is empty.

Unbuffered channels synchronize; buffered channels decouple. Start with unbuffered and add a buffer only when you can say why.

RULE OF THUMB

Default to unbuffered channels. Their blocking behaviour makes the flow of your program visible and deadlocks loud. Add a buffer when you have a measured reason: smoothing bursts between a fast producer and a slow consumer, or when the buffer size encodes a real limit (a semaphore). A buffer never fixes a design problem; it hides it until the buffer fills.

Channel direction in signatures

```
func produce(out chan<- Job) { /* can only send */ }
func consume(in <-chan Job) { /* can only receive */ }
```

Declaring direction on parameters lets the compiler enforce who sends and who receives, and documents the data flow. A bidirectional channel converts implicitly to either direction.

Closing: the rules

Only the **sender** closes a channel, and only when it will send no more. Closing signals “no more values” to every receiver, which makes `range` loops end. Receivers never close. Closing twice panics; sending on a closed channel panics; receiving from a closed channel returns the zero value immediately, forever. If several goroutines send on one channel, none of them can know when to close it, so a coordinator waits for all of them (with a `WaitGroup`) and then closes (Chapter 15). You don’t need to close a channel you’re done with; the garbage collector reclaims it. Close only to communicate.

select: waiting on several channels

`select` blocks until one of its cases can proceed, choosing randomly if several are ready:

```
select {
case job := <-jobs:
    handle(job)
case <-ctx.Done():           // cancellation (Chapter 14)
    return ctx.Err()
case <-time.After(2 * time.Second):
    return errors.New("timed out waiting for a job")
}

select {                      // non-blocking with default
case results <- r:
default:
    log.Println("results channel full, dropping")
}
```

Three idioms are built from this: **timeouts** (`time.After` or a context deadline), **cancellation** (a done channel or `ctx.Done()` case in every loop), and **non-blocking sends and receives** (a `default` case). A `nil` channel blocks forever in a `select`, which is a trick for disabling a case: set the channel variable to `nil` once it’s drained.

GOTCHA

`time.After` inside a loop creates a new timer each iteration that isn’t collected until it fires. For a per-iteration timeout use `time.NewTimer` and `Stop` it, or better, derive a context with `context.WithTimeout`. For a total deadline across the whole loop, create the timer or context *once*, outside the loop.

When to use a channel

Channels shine when you are **handing off ownership of data** (jobs to workers, results back), **signalling events** (done, cancel, tick), or **coordinating a pipeline**. They are a poor fit for protecting shared state that many goroutines read and write, such as a cache or a counter. That is what mutexes are for, and the next chapter draws the line.

Self-check

1. What is the difference in behaviour between `make(chan int)` and `make(chan int, 1)` when a goroutine sends once and nobody receives?
2. Who closes a channel, and what happens to a `range` loop when it is closed?
3. Write, from memory, a `select` that reads from a results channel or gives up after five seconds.

CHAPTER 13

Sharing memory safely: sync and atomic

Channels are for moving data between goroutines. Mutexes are for guarding data that stays in one place. Most real programs use both, and knowing which to reach for is the concurrency skill that matters most.

sync.Mutex

```
type Counter struct {
    mu      sync.Mutex
    counts map[string]int
}

func (c *Counter) Inc(key string) {
    c.mu.Lock()
    defer c.mu.Unlock()
    c.counts[key]++
}

func (c *Counter) Snapshot() map[string]int {
    c.mu.Lock()
    defer c.mu.Unlock()
    return maps.Clone(c.counts) // never hand out the guarded map itself
}
```

The pattern: put the mutex *next to the data it protects*, in the same struct, and make every access to that data go through methods that lock. `defer Unlock()` right after `Lock()` guarantees release on every path, including panics. Keep the locked region as short as possible and never do I/O or call unknown code while holding a lock. A `Mutex` must not be copied after use (go vet reports this as `copylocks`), which is one more reason for pointer receivers.

sync.RWMutex

When reads vastly outnumber writes, `RWMutex` lets many readers proceed simultaneously while a writer gets exclusive access: `RLock`/`RUnlock` for reads, `Lock`/`Unlock` for writes. It has more overhead than `Mutex`; use it only when contention is measured, and default to `Mutex`.

sync.Once

```
var (
    once      sync.Once
    templates *template.Template
)

func templates() *template.Template {
    once.Do(func() {
        templates = template.Must(template.ParseFS(webFS, "templates/*.html"))
    })
    return templates
}
```

`Once.Do` runs its function exactly once even when called concurrently; every other caller blocks until the first finishes. It is the correct way to lazily initialize something expensive.

`sync.OnceValue` (Go 1.21+) wraps a function that returns a value for the same effect with less ceremony.

sync/atomic

For a single numeric counter or flag, a mutex is overkill:

```
var requests atomic.Int64
requests.Add(1)
n := requests.Load()
```

The typed atomics (`atomic.Int64`, `atomic.Bool`, `atomic.Pointer[T]`) are the modern API. Use them for metrics-style counters and “has this happened yet” flags. The moment you need to update two things consistently, go back to a mutex.

Channels or mutex?

SITUATION	REACH FOR
Handing work items to a pool of workers	Channel
Collecting results from many goroutines	Channel, or an indexed slice + <code>WaitGroup</code>
Signalling “stop” or “done”	<code>context</code> (a channel underneath)
A cache, registry or counter read and written by many goroutines	<code>Mutex</code> inside the struct
One-time lazy initialization	<code>sync.Once</code>
A single counter or flag	<code>atomic</code>
Limiting concurrency to N	Buffered channel as a semaphore, or <code>errgroup.SetLimit</code>

RULE OF THUMB

If the data has an owner that changes over time, use a channel to pass it along. If the data stays put and many goroutines visit it, use a mutex. If you find yourself building a mutex out of channels, or a queue out of mutexes, you've picked the wrong tool.

Self-check

1. Why should `Snapshot` return a clone rather than the map itself?
2. What does `go vet` mean by `copylocks`, and how do pointer receivers prevent it?
3. A struct has a `hits int` field incremented from many goroutines. Give two correct fixes.

CHAPTER 14

Cancellation with context

Because a goroutine can't be killed, every long-running operation in Go accepts a `context.Context` and stops when it is cancelled. Contexts carry deadlines and cancellation through call trees, from an HTTP request all the way down to a database query.

The API

```
ctx := context.Background()           // root; never cancelled; for main
and tests
ctx, cancel := context.WithCancel(parent) // cancel() cancels ctx and its
children
ctx, cancel := context.WithTimeout(parent, 3*time.Second)
ctx, cancel := context.WithDeadline(parent, time.Now().Add(time.Minute))
defer cancel()                         // always; frees resources even on
success

<-ctx.Done()                          // channel closed when cancelled or expired
ctx.Err()                             // nil until Done; then Canceled or DeadlineExceeded
```

Cancelling a parent cancels every context derived from it. A child cannot cancel its parent. That tree shape is the whole design.

The conventions

1. `ctx` is the **first parameter** of any function that blocks, does I/O, or may run long: `func fetch(ctx context.Context, url string) error`.
2. **Never store a context in a struct**; pass it through calls. The exception is `http.Request`, which carries one you get with `r.Context()`.
3. Every loop that could run for a while **checks `ctx.Done()`** in a `select`, or calls something that does.
4. **Call `cancel()`** via `defer` the moment you create a cancellable context. Forgetting leaks the timer and the goroutine behind it (go vet reports “the cancel function is not used”).
5. Use `ctx.Value` only for request-scoped metadata that crosses API boundaries, such as a request ID or an authenticated user. Never for passing function parameters.

Where contexts come from in practice

```
// HTTP server: the request's context is cancelled when the client disconnects.
func handler(w http.ResponseWriter, r *http.Request) {
    ctx, cancel := context.WithTimeout(r.Context(), 5*time.Second)
    defer cancel()
    rows, err := db.QueryContext(ctx, "SELECT ...")
    // ...
}

// HTTP client: bound how long a request may take.
req, err := http.NewRequestWithContext(ctx, http.MethodGet, url, nil)
resp, err := client.Do(req)

// A program that stops cleanly on Ctrl-C.
ctx, stop := signal.NotifyContext(context.Background(), os.Interrupt, syscall.SIGTERM)
defer stop()
```

Standard-library functions that block take a context or have a `...Context` variant:

`db.QueryContext`, `exec.CommandContext`, `net.Dialer.DialContext`, time-based waits via `select` on `ctx.Done()`. Any function of yours that calls one of these takes a `ctx` too, and passes it down unchanged.

Sleeping with a context

`time.Sleep` cannot be interrupted. In a goroutine that should stop on cancellation, write:

```
select {
case <-time.After(d):           // fine outside a loop; inside one, use a Timer or Ticker
case <-ctx.Done():
    return ctx.Err()
}
```

Self-check

1. Why does the linter complain when you discard the `cancel` function?
2. What is `ctx.Err()` after a `WithTimeout` context expires? After `cancel()` is called?
3. An HTTP handler starts a goroutine that outlives the request. Which context must that goroutine not use, and why?

CHAPTER 15

The five patterns that cover 80% of concurrent code

Almost every concurrent Go program is a composition of five patterns. Learn them as fixed shapes, the way you learned `for` loops, and you will rarely write a novel concurrent structure.

Pattern 1: fan-out with a bounded worker pool

Send jobs on a channel; N workers receive them; results come back on another channel; a closer goroutine closes the results channel when all workers exit. This is the skeleton of Project 2.

```
func checkAll(ctx context.Context, urls []string, workers int) []Result {
    jobs := make(chan string)
    results := make(chan Result)

    var wg sync.WaitGroup
    for range workers {
        wg.Add(1)
        go func() {
            defer wg.Done()
            for url := range jobs {           // exits when jobs is closed
                results <- check(ctx, url)
            }
        }()
    }

    go func() {                               // producer
        defer close(jobs)                     // tell workers there is no more work
        for _, u := range urls {
            select {
            case jobs <- u:
            case <- ctx.Done():
                return
            }
        }
    }()

    go func() {                               // closer: results is closed once, here
        wg.Wait()
        close(results)
    }()

    var out []Result
    for r := range results {                  // main goroutine consumes until closed
        out = append(out, r)
    }
    return out
}
```

Read it twice and trace the shutdown: producer closes `jobs`; each worker's `range` ends and it calls `Done`; the closer's `Wait` returns and closes `results`; the consumer's `range` ends. No channel is closed by a receiver, nothing leaks, and cancellation stops the producer early.

Pattern 2: a semaphore for “at most N at a time”

When you already have one goroutine per task and only want to bound how many run simultaneously:

```
sem := make(chan struct{}, 10) // capacity is the concurrency limit
var wg sync.WaitGroup
for _, u := range urls {
    wg.Add(1)
    go func() {
        defer wg.Done()
        sem <- struct{}{} // acquire (blocks when 10 are running)
        defer func() { <-sem }() // release
        check(ctx, u)
    }()
}
wg.Wait()
```

`struct{}` is a zero-size type; a `chan struct{}` carries no data, only signals. Simpler than a worker pool when tasks are known up front; use the pool when tasks arrive over time.

Pattern 3: `errgroup` for “run these, stop on first error”

golang.org/x/sync/errgroup (the one external module worth learning early) wraps `WaitGroup`, first-error capture, cancellation and a concurrency limit:

```
g, ctx := errgroup.WithContext(ctx)
g.SetLimit(10)
for _, u := range urls {
    g.Go(func() error {
        return check(ctx, u) // a non-nil error cancels ctx for the others
    })
}
if err := g.Wait(); err != nil { // first error, or nil
    return err
}
```

Install with `go get golang.org/x/sync/errgroup`. Use it whenever “all must succeed” is the semantics; use Pattern 1 when you want every result, including failures.

Pattern 4: a periodic worker with clean shutdown

Background jobs (polling, cleanup, health checks) are a `Ticker` loop with a context case. This is the heart of Project 4.

```
func runMonitor(ctx context.Context, every time.Duration, tick func(context.Context)) {
    t := time.NewTicker(every)
    defer t.Stop()
    tick(ctx) // run once immediately, then on each tick
    for {
        select {
        case <-t.C:
            tick(ctx)
        case <-ctx.Done():
            return
        }
    }
}
```

Pattern 5: pipeline stages

Each stage is a function that receives from one channel and sends to another; stages run concurrently and process items as they flow:

```
func parse(ctx context.Context, lines <-chan string) <-chan Record {
    out := make(chan Record)
    go func() {
        defer close(out)
        for l := range lines {
            r, err := parseLine(l)
            if err != nil {
                continue
            }
            select {
            case out <- r:
            case <-ctx.Done():
                return
            }
        }
    }()
    return out
}
```

The stage owns its output channel and closes it; the caller composes stages: `for r := range parse(ctx, read(ctx, f))`. Add a worker pool inside a stage when that stage is the bottleneck.

Combining them: what a real service looks like

An HTTP server (its own goroutine per request, handled by `net/http`), a monitor loop (Pattern 4) that fans out checks with a limit (Pattern 3), writing results under a mutex (Chapter 13), all cancelled by one `signal.NotifyContext` on shutdown. That is Project 4, and every piece is one of the shapes above.

Self-check

1. In Pattern 1, why is `results` closed by a separate goroutine that waits on the WaitGroup rather than by the workers?

2. What goes wrong in Pattern 2 if you acquire the semaphore *before* the `go` statement? (Hint: nothing breaks, but something changes.)
3. Why does Pattern 4 call `tick` once before entering the loop?

CHAPTER 16

Pitfalls and the race detector

Concurrency bugs in Go are few in kind and fully predictable. Here are the ones you will write, what they look like, and the tool that finds most of them.

The race detector

Build or test with `-race` and the runtime instruments every memory access, reporting when two goroutines touch the same location without synchronization and at least one writes:

```
=====
WARNING: DATA RACE
Write at 0x00c000126018 by goroutine 7:
    main.(*Counter).Inc()
        /home/you/linkcheck/counter.go:14 +0x64
Previous read at 0x00c000126018 by goroutine 6:
    main.(*Counter).Snapshot()
        /home/you/linkcheck/counter.go:21 +0x3c
=====
```

It tells you both access sites with line numbers. It only catches races that actually occur during the run, so write tests that exercise concurrency and run them with `go test -race` every time. A data race is undefined behaviour: the program may “work” today and corrupt memory tomorrow. Zero races is the bar, not “it seems fine”.

The catalogue

SYMPTOM	CAUSE	FIX
<code>fatal error: all goroutines are asleep - deadlock!</code>	Every goroutine is blocked: typically a send with no receiver, or <code>wg.Wait()</code> with a missing <code>Done</code> , or waiting on a channel nobody will close	Trace who is supposed to receive or close; run the shutdown sequence in your head as in Pattern 1
Program hangs but no deadlock message	Some goroutine is still runnable (a ticker, a listener), so the runtime doesn't detect it	Same analysis; add a <code>select</code> with <code>ctx.Done()</code> to every blocking operation
<code>panic: send on closed channel</code>	Producer closed too early, or a receiver closed	Only the sole sender closes, after its last send; with many senders, a coordinator closes after <code>wg.Wait()</code>
<code>panic: close of closed channel</code>	Two goroutines both think they own the channel	Assign one owner; use <code>sync.Once</code> if unavoidable
<code>fatal error: concurrent map writes</code>	A map written from several goroutines	Mutex in the struct that owns the map; this fatal error cannot be recovered

SYMPTOM	CAUSE	FIX
Goroutine leak (memory grows, <code>runtime.NumGoroutine()</code> climbs)	A goroutine blocked forever on a send or receive nobody will complete, often after the consumer gave up on a timeout	Every send/receive in a long-lived goroutine is inside a <code>select</code> with <code>ctx.Done()</code> ; the consumer that times out also cancels the producer
<code>WaitGroup</code> is reused before previous <code>Wait</code> has returned or negative counter panic	<code>Add</code> inside the goroutine, or more <code>Done</code> than <code>Add</code>	<code>Add</code> in the parent before <code>go</code> ; exactly one <code>defer wg.Done()</code> per goroutine
Results appear in random order	Channels don't preserve producer order across workers	Include an index in the result and sort, or use the indexed-slice pattern from Chapter 11
<code>go vet: copylocks</code>	A struct containing a <code>Mutex</code> was copied (value receiver, assigned, passed by value)	Pointer receivers; pass <code>*T</code>
Wrong value captured in a goroutine	Closure over a loop variable in Go <1.22, or over a variable that changes later	Update Go; pass the value as a parameter to the goroutine's function
Timer or context leak	<code>time.After</code> in a hot loop; cancel functions never called	<code>NewTimer/Stop</code> ; <code>defer cancel()</code>

Habits that prevent most of this

- Every goroutine you start has a known way to stop and a known place where you wait for it. If you can't name both, don't start it yet.
- Every blocking channel operation in a long-lived goroutine sits inside a `select` with a cancellation case.
- Shared state lives in a struct with its mutex; nothing outside the struct touches the fields.
- Tests for concurrent code run with `-race`, and at least one test starts many goroutines against the shared component.
- When in doubt, simplify: fewer goroutines, one channel, or a mutex. Clever concurrency is a liability.

VERSION NOTE

Go 1.27 promoted the `goroutineleak` profile in `runtime/pprof` to generally available: it detects goroutines that are permanently blocked. For this book, `runtime.NumGoroutine()` printed before and after a test is enough to catch leaks in Project 2.

Self-check

1. Why doesn't a passing test suite without `-race` prove the absence of data races?
2. Sketch a goroutine leak in five lines, then fix it with a `select`.
3. What single ownership rule prevents both "send on closed channel" and "close of closed channel"?

PROJECT 2

linkcheck, a concurrent link checker

linkcheck • check hundreds of URLs in parallel, with timeouts, limits and a clean shutdown

Time box: 6–8 hours over 3 sessions • **Chapters needed:** 11–16 (plus 5–8, 10) • **Dependencies:** none required; `golang.org/x/sync/errgroup` optional

You will exercise: goroutines, channels, `select`, `sync.WaitGroup`, `sync.Mutex`, `context` timeouts and cancellation, the `net/http` client, `httptest.NewServer`, the race detector

Why this project

A link checker is I/O-bound, embarrassingly parallel, and full of real-world failure modes: slow servers, connection refusals, redirects, DNS errors. Sequentially it takes minutes; with a worker pool it takes seconds, and you will see the speed-up in your terminal, which is the moment goroutines stop being abstract. It also has a natural place for every concurrency primitive, and it must shut down cleanly on Ctrl-C.

What it looks like when finished

```
$ linkcheck -file urls.txt -workers 20 -timeout 5s
200    142ms https://go.dev/
200     98ms https://pkg.go.dev/net/http
301    210ms https://golang.org/doc -> https://go.dev/doc
404    187ms https://go.dev/this-does-not-exist
ERR     5.0s https://10.255.255.1/ context deadline exceeded
ERR     31ms https://nope.invalid/ dial tcp: lookup nope.invalid: no such host

checked 6 urls in 5.02s: 3 ok, 1 redirect, 1 broken, 1 error
$ echo $?
1
```

Lines print as results arrive (not in input order), the summary prints once at the end, and the exit code is 1 if anything is broken or errored. With `-crawl https://example.com` (a stretch goal) it fetches the page and checks every link on it.

Suggested layout

```
linkcheck/
go.mod
main.go      // flags, signal handling, run()
check.go     // Result type, check(ctx, client, url) Result
pool.go      // checkAll(ctx, urls, workers, onResult) Summary
report.go    // formatting a Result line and the Summary
check_test.go // uses httpptest.NewServer
pool_test.go // runs under -race
LEARNING.md
```

Milestones

M1 Check one URL correctly (1.5 h)

Define `Result{URL, Status int, Latency time.Duration, RedirectTo string, Err error}`. Write `check(ctx, client, url)`: build the request with `http.NewRequestWithContext`, issue it with a shared `*http.Client` whose `CheckRedirect` returns `http.ErrUseLastResponse` so you observe 3xx yourself, read the `Location` header, and **always** `resp.Body.Close()` (drain it with `io.Copy(io.Discard, resp.Body)` first so the connection can be reused). Measure latency with `time.Since`.

Accept when: a table-driven test using `httpptest.NewServer` covers 200, 404, 301 (with `Location`), a handler that sleeps longer than the timeout (expect `context.DeadlineExceeded` via `errors.Is`), and an unreachable address. Use `-timeout` per request via `context.WithTimeout` derived from the parent context.

M2 The worker pool (2 h)

Implement Pattern 1 from Chapter 15: a `jobs` channel fed by a producer goroutine, `N` workers, a `results` channel closed by a coordinator after `wg.Wait()`. The consumer calls an `onResult(Result)` callback for each result as it arrives and accumulates a `Summary` (counts by category, total elapsed). The producer must stop early when the context is cancelled.

Accept when: `go test -race` passes a test that checks 200 URLs against a single `httpptest.Server` with 10 workers and verifies every URL appears exactly once in the results (use a `map[string]int` guarded by a mutex inside the test callback). Assert that `runtime.NumGoroutine()` after `checkAll` returns is back to its pre-call value (allow a small delay); this proves no leaks. Time the run with 1 worker and with 20 workers against a handler that sleeps 50 ms per request; the 20-worker run must be at least 10× faster.

M3 CLI, signals and reporting (1.5 h)

Flags: `-file` (default stdin, one URL per line, skip blanks and `#` comments), `-workers` (default 10), `-timeout` (a `time.Duration`, default 10s). Create the root context with `signal.NotifyContext(context.Background(), os.Interrupt)`. Print each result as it arrives in the fixed-width format above, then the summary, then exit 1 if any result is 4xx/5xx or an error. On Ctrl-C the program stops within one timeout period, prints the summary for what completed, and exits 130.

Accept when: the transcript above reproduces against real URLs of your choosing; pressing Ctrl-C mid-run produces a partial summary and no `panic` or deadlock; `go vet` is clean.

M4 Rate limit per host (1 h)

Real sites throttle aggressive clients. Add a per-host limiter: a `map[string]chan struct{}` of semaphores (capacity `-per-host`, default 2) guarded by a mutex, so no more than N requests hit one host concurrently even with 50 workers. Extract the host with `net/url`.

Accept when: a test with two fake servers and `-per-host 1` shows (via timestamps recorded in the handlers) that requests to the same host never overlap while requests to different hosts do.

Definition of done

- ☐ All milestones pass under `go test -race -count=1 ./...`.
- ☐ Every goroutine you start has a documented stop condition (write it as a comment above the `go` statement).
- ☐ No channel is closed by a receiver; no send happens outside a `select` in any long-lived goroutine.
- ☐ You can draw the goroutine and channel topology on paper and narrate the shutdown sequence in order.
- ☐ You can explain why the results arrive out of order and how you would preserve order if a user asked for it.
- ☐ Commit tagged `done`; `LEARNING.md` updated; optional AI review only after this point.

Stretch goals (optional, pick one)

- `-crawl URL`: fetch a page and extract `href` values with `golang.org/x/net/html` (or a simple regex to stay dependency-free); resolve relative links with `url.ResolveReference`.
- Retry transient failures (5xx, timeouts) once with a short backoff, without blocking the worker for other jobs.
- Replace the hand-rolled pool with `errgroup` and `SetLimit`, then explain in `LEARNING.md` which version you prefer and why.
- `-json` output, one object per line, suitable for `jq`.

PART IV

Building real services

The standard library contains a production HTTP server, a templating engine, a JSON codec and a database layer. Four chapters show how they fit together, and Project 3 has you ship a REST API with an HTML frontend and a SQLite database as a single binary.

Chapter 17 · The `net/http` server

Chapter 18 · An HTML frontend: templates, static files, embed

Chapter 19 · Persistence with `database/sql` and SQLite

Chapter 20 · Configuration, logging and shipping one binary

Project 3 · `shelf`, a reading-list API with a web UI

CHAPTER 17

The net/http server

In Go, the standard library is the web framework. `net/http` gives you a router, a production server, and a handler abstraction so simple that most third-party frameworks are thin layers over it. Learn it directly and you'll understand all of them.

Handlers

Everything is an `http.Handler`:

```
type Handler interface {
    ServeHTTP(w http.ResponseWriter, r *http.Request)
}
```

`http.HandlerFunc` adapts an ordinary function to that interface, so a handler is usually just a function. `w` is an `io.Writer` (plus headers and status); `r` carries the method, URL, headers, body and a context.

```
func hello(w http.ResponseWriter, r *http.Request) {
    name := r.URL.Query().Get("name") // ?name=ada
    if name == "" {
        name = "world"
    }
    fmt.Fprintf(w, "hello, %s\n", name) // status 200 is implied on first write
}
```

Routing with ServeMux (Go 1.22+)

```
mux := http.NewServeMux()
mux.HandleFunc("GET /{$}", s.handleIndex)           // exactly "/"
mux.HandleFunc("GET /api/items", s.handleListItems)
mux.HandleFunc("POST /api/items", s.handleCreateItem)
mux.HandleFunc("GET /api/items/{id}", s.handleGetItem)
mux.HandleFunc("PATCH /api/items/{id}", s.handleUpdateItem)
mux.HandleFunc("DELETE /api/items/{id}", s.handleDeleteItem)
mux.Handle("GET /static/", http.FileServerFS(staticFS))

id := r.PathValue("id")                             // inside a handler
```

Patterns are `[METHOD] [HOST]/path`. `{id}` captures one segment; `{rest...}` captures everything remaining; `{$}` anchors an exact match (without it, `/` matches every path). Most specific pattern wins; a method mismatch yields 405 automatically. That is all the routing most services need.

VERSION NOTE

Method-and-wildcard patterns arrived in Go 1.22. Older tutorials use `mux.HandleFunc("/items/", ...)` and switch on `r.Method` by hand, or reach for `chi` or `gorilla/mux`. You don't need any of that now.

Handlers with dependencies

Handlers need a store, a logger, templates. Don't use globals; hang the handlers off a struct:

```
type Server struct {
    store Store           // an interface defined in this package
    log    *slog.Logger
    tpl    *template.Template
}

func (s *Server) handleGetItem(w http.ResponseWriter, r *http.Request) {
    id, err := strconv.ParseInt(r.PathValue("id"), 10, 64)
    if err != nil {
        writeError(w, http.StatusBadRequest, "invalid id")
        return
    }
    item, err := s.store.Get(r.Context(), id)
    switch {
    case errors.Is(err, ErrNotFound):
        writeError(w, http.StatusNotFound, "item not found")
        return
    case err != nil:
        s.log.Error("get item", "id", id, "err", err)
        writeError(w, http.StatusInternalServerError, "internal error")
        return
    }
    writeJSON(w, http.StatusOK, item)
}
```

Note the shape: parse and validate input, call the store with the request's context, map errors to status codes, write the response. Never leak internal error text to clients; log it and return a generic message.

JSON in and out

Two helpers you will write once and reuse in every project:

```
func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json")
    w.WriteHeader(status)           // headers must be set before this
    if err := json.NewEncoder(w).Encode(v); err != nil {
        // too late to change the status; log it
    }
}

func writeError(w http.ResponseWriter, status int, msg string) {
    writeJSON(w, status, map[string]string{"error": msg})
}

func decodeJSON(w http.ResponseWriter, r *http.Request, dst any) error {
    r.Body = http.MaxBytesReader(w, r.Body, 1<<20) // refuse bodies over 1 MiB
    dec := json.NewDecoder(r.Body)
    dec.DisallowUnknownFields()                  // typos in field names become errors
    return dec.Decode(dst)
}
```

Decode into a dedicated request struct (`createItemRequest`) rather than your domain type, so clients can't set fields like `ID` or `CreatedAt`. Use pointer fields (`*string`) in a PATCH request struct to distinguish “not provided” from “empty”.

Middleware

Middleware is a function that takes a handler and returns a handler. The pattern falls out of the interface:

```
func logging(log *slog.Logger) func(http.Handler) http.Handler {
    return func(next http.Handler) http.Handler {
        return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
            start := time.Now()
            next.ServeHTTP(w, r)
            log.Info("request", "method", r.Method, "path", r.URL.Path,
                "dur", time.Since(start))
        })
    }
}

func recoverPanic(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        defer func() {
            if rec := recover(); rec != nil {
                http.Error(w, "internal error", http.StatusInternalServerError)
            }
        }()
        next.ServeHTTP(w, r)
    })
}

var handler http.Handler = mux
handler = logging(log)(recoverPanic(handler)) // outermost runs first
```

To record the status code in the logger you wrap `ResponseWriter` in a small struct that captures `WriteHeader`; you will do that in Project 3. Beyond logging and recovery, common middleware includes request IDs, authentication, and CORS (which you won't need when the frontend is served from the same origin).

The server itself

```

srv := &http.Server{
    Addr:           addr,
    Handler:        handler,
    ReadHeaderTimeout: 5 * time.Second,      // defends against slow-loris clients
    ReadTimeout:    10 * time.Second,
    WriteTimeout:   30 * time.Second,
    IdleTimeout:    60 * time.Second,
}

go func() {
    if err := srv.ListenAndServe(); err != nil && !errors.Is(err, http.ErrServerClosed) {
        log.Error("server", "err", err)
        os.Exit(1)
    }
}()

<-ctx.Done() // signal.NotifyContext from Chapter 14
shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
defer cancel()
if err := srv.Shutdown(shutdownCtx); err != nil { /* log */ }

```

`http.ListenAndServe(addr, h)` with no timeouts is fine for a five-minute experiment and wrong for anything else. `Shutdown` stops accepting connections, waits for in-flight requests to finish (up to the deadline), and makes `ListenAndServe` return `http.ErrServerClosed`, which is not an error to you. Every request already runs in its own goroutine; you never write `go` in a handler for the request itself.

Testing handlers

```

func TestGetItemNotFound(t *testing.T) {
    s := NewServer(NewMemStore(), slog.New(slog.NewTextHandler(io.Discard, nil)))
    req := httptest.NewRequest(http.MethodGet, "/api/items/42", nil)
    rec := httptest.NewRecorder()
    s.routes().ServeHTTP(rec, req) // routes() returns the mux with middleware
    if rec.Code != http.StatusNotFound {
        t.Fatalf("status = %d, want 404; body: %s", rec.Code, rec.Body)
    }
}

```

No network, no port, microseconds per test. For a request with a body, pass a `strings.NewReader` holding the JSON and set `Content-Type`. For end-to-end tests including the real listener, `httptest.NewServer(handler)` gives you a URL.

Self-check

1. Why must `w.Header().Set` happen before `w.WriteHeader`, and what happens if you call `WriteHeader` twice?
2. Write the `logging` middleware from memory, then extend it to capture the status code.
3. What does `Shutdown` do that simply exiting the process does not?

CHAPTER 18

An HTML frontend: templates, static files, embed

You want to see a page in the browser talking to your API. Go gives you two complementary tools: server-rendered HTML through `html/template`, and static files (including a small JavaScript app) compiled into the binary with `embed`. Project 3 uses both.

html/template

`html/template` has the same API as `text/template` but escapes output contextually: a value inserted into HTML text, an attribute, a URL or a script gets the right escaping automatically. This is why you use it instead of string concatenation, and why Go templates are safe by default against injection.

```

<!-- web/templates/index.html -->
{{define "index"}}
<!doctype html>
<html><head><meta charset="utf-8"><title>shelf</title>
<link rel="stylesheet" href="/static/style.css"></head>
<body>
  <h1>Reading list ({{len .Items}})</h1>
  <ul id="items">
    {{range .Items}}
      <li data-id="{{.ID}}" class="{{.Status}}">
        <a href="{{.URL}}">{{.Title}}</a>
        {{if .Tags}}<span class="tags">{{range .Tags}}#{{.}} {{end}}</span>{{end}}
      </li>
    {{else}}
      <li class="empty">Nothing here yet.</li>
    {{end}}
  </ul>
  <script src="/static/app.js"></script>
</body></html>
{{end}}

```

`.` is the current value: the data you passed at the top level, each element inside `range`. `{{if}}`, `{{range}}` / `{{else}}`, `{{with}}`, pipelines (`{{.Title | printf "%q"}}`) and template calls (`{{template "nav" .}}`) cover what you need. Functions you want available (formatting dates, for instance) are registered with `Funcs` before parsing.

Parse templates once at start-up, not per request, and fail fast if they're broken:

```

tpl := template.Must(template.ParseFS(webFS, "templates/*.html"))

func (s *Server) handleIndex(w http.ResponseWriter, r *http.Request) {
    items, err := s.store.List(r.Context(), ListFilter{})
    if err != nil { /* 500 */ }
    var buf bytes.Buffer // render to a buffer first...
    data := map[string]any{"Items": items}
    if err := s.tpl.ExecuteTemplate(&buf, "index", data); err != nil {
        s.log.Error("render", "err", err)
        http.Error(w, "template error", http.StatusInternalServerError)
        return
    }
    w.Header().Set("Content-Type", "text/html; charset=utf-8")
    buf.WriteTo(w) // ...so a mid-render error never sends half a
page
}

```

Static files and embed

```

//go:embed templates static
var webFS embed.FS

```

The directive above a variable of type `embed.FS` compiles those directories into the binary. Serve them with `http.FileServerFS(webFS)` behind a `GET /static/` pattern; the request path `/static/app.js` maps to the embedded file `static/app.js`. Your deliverable stays a single file that you can copy anywhere. During development, restart with `go run ./cmd/shelf` after editing templates; if that bothers you, load from disk when a `-dev` flag is set (using `os.DirFS("web")`), which also satisfies `fs.FS`) and from `embed.FS` otherwise. Both implement the same interface, which is the point.

A minimal JavaScript client

The frontend of Project 3 is deliberately plain: no bundler, no framework, one `app.js` that calls the JSON API with `fetch`. Because the HTML and the API are served from the same origin, there is no CORS to configure.

```
// web/static/app.js
async function addItem(url, title) {
  const res = await fetch("/api/items", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify({url, title}),
  });
  if (!res.ok) {
    const {error} = await res.json();
    throw new Error(error);
  }
  return res.json();
}

document.querySelector("#add-form").addEventListener("submit", async (e) => {
  e.preventDefault();
  const form = new FormData(e.target);
  try {
    await addItem(form.get("url"), form.get("title"));
    location.reload(); // simplest refresh; upgrade to DOM updates later
  } catch (err) {
    document.querySelector("#error").textContent = err.message;
  }
});
```

This is the whole “wire an API to an HTML frontend and see it work” loop. You already know this side; the point is to feel how little ceremony the Go side needs to serve it.

Server-rendered or API-driven?

Do both, as Project 3 does: render the initial page on the server (fast first paint, works without JavaScript, trivially testable with `httptest`), and use the JSON API for interactions. If you later want live updates, server-sent events (Project 4) fit the `net/http` model perfectly: a handler that flushes lines to `w` as events happen.

Self-check

1. What does `html/template` do that `text/template` does not, and why does that matter for a field a user typed?
2. Why render into a buffer before writing to the `ResponseWriter`?
3. Explain how the same handler can serve files from disk in development and from the binary in production.

CHAPTER 19

Persistence with database/sql and SQLite

Go's standard database layer is small and explicit: you write SQL, you scan rows into structs, you handle errors. No ORM is needed for the projects in this book, and understanding `database/sql` directly means every driver and helper library later will be obvious.

Choosing SQLite and a driver

SQLite is the right database for a local, single-binary service: zero configuration, one file, transactional, fast. `database/sql` is driver-agnostic; you import a driver for its side effect of registering itself:

```
import (
    "database/sql"
    _ "modernc.org/sqlite" // pure-Go driver, no C compiler needed; name "sqlite"
)

db, err := sql.Open("sqlite", "file:shelf.db")
if err != nil { return err }
db.SetMaxOpenConns(1) // SQLite serializes writers anyway; avoids "database is locked"
if _, err := db.ExecContext(ctx, "PRAGMA journal_mode=WAL; PRAGMA foreign_keys=ON;");
err != nil {
    return err
}
if err := db.PingContext(ctx); err != nil { // Open is lazy; Ping actually connects
    return err
}
```

Install with `go get modernc.org/sqlite`. The alternative, `github.com/mattn/go-sqlite3`, is the classic driver but needs `cgo` and a C toolchain (driver name `sqlite3`); prefer the pure-Go one for a learning project. `*sql.DB` is a connection pool, safe for concurrent use, created once and shared; never open it per request.

Migrations, the simple way

Keep schema changes as numbered SQL files, embed them, and apply the ones not yet applied at start-up:

```
-- migrations/001_items.sql
CREATE TABLE IF NOT EXISTS items (
    id          INTEGER PRIMARY KEY,
    url         TEXT NOT NULL,
    title       TEXT NOT NULL,
    notes       TEXT NOT NULL DEFAULT '',
    status      TEXT NOT NULL DEFAULT 'unread',
    tags        TEXT NOT NULL DEFAULT '[]', -- JSON array; good enough here
    added_at    TEXT NOT NULL,             -- RFC 3339
    finished_at TEXT
);
```

Track the applied version in a one-row `schema_version` table (or SQLite's `PRAGMA user_version`), read it, and execute each file with a higher number inside a transaction. Thirty lines of Go, no library. You will write it in Project 3, M4.

Queries

```
func (s *SQLiteStore) Get(ctx context.Context, id int64) (Item, error) {
    var it Item
    var tags, addedAt string
    var finishedAt sql.NullString
    err := s.db.QueryRowContext(ctx,
        `SELECT id, url, title, notes, status, tags, added_at, finished_at
        FROM items WHERE id = ?`, id,
    ).Scan(&it.ID, &it.URL, &it.Title, &it.Notes, &it.Status,
        &tags, &addedAt, &finishedAt)
    if errors.Is(err, sql.ErrNoRows) {
        return Item{}, ErrNotFound // translate driver errors into domain errors here
    }
    if err != nil {
        return Item{}, fmt.Errorf("get item %d: %w", id, err)
    }
    if err := json.Unmarshal([]byte(tags), &it.Tags); err != nil { return Item{}, err }
    if it.AddedAt, err = time.Parse(time.RFC3339, addedAt); err != nil {
        return Item{}, err
    }
    if finishedAt.Valid {
        t, err := time.Parse(time.RFC3339, finishedAt.String)
        if err != nil { return Item{}, err }
        it.FinishedAt = &t
    }
    return it, nil
}

func (s *SQLiteStore) List(ctx context.Context, f ListFilter) ([]Item, error) {
    rows, err := s.db.QueryContext(ctx,
        `SELECT ... FROM items WHERE (? = '' OR status = ?) ORDER BY added_at DESC`,
        f.Status, f.Status)
    if err != nil { return nil, err }
    defer rows.Close() // always; releases the connection
    var out []Item
    for rows.Next() {
        var it Item
        if err := rows.Scan(/* same columns */); err != nil { return nil, err }
        out = append(out, it)
    }
    return out, rows.Err() // an error during iteration shows up here
}

func (s *SQLiteStore) Add(ctx context.Context, it Item) (Item, error) {
    res, err := s.db.ExecContext(ctx,
        `INSERT INTO items (url, title, notes, status, tags, added_at) VALUES
        (?, ?, ?, ?, ?, ?)`,
        it.URL, it.Title, it.Notes, it.Status, tagsJSON, it.AddedAt.Format(time.RFC3339))
    if err != nil { return Item{}, fmt.Errorf("insert item: %w", err) }
    it.ID, err = res.LastInsertId()
    return it, err
}
```

The rules: always the `Context` variants; always `?` placeholders (never `fmt.Sprintf` into SQL: that is how injection happens); always `defer rows.Close()` and check `rows.Err()`; `QueryRow` for exactly one row, `Query` for many, `Exec` for statements that return no rows. `sql.Null*` types or pointers handle nullable columns. Store times as RFC 3339 text in SQLite and parse on read; it sorts correctly and stays readable in any SQLite browser.

The `Scan` boilerplate is repetitive. That is the honest cost of no ORM; a small `scanItem(rows)` helper shared by `Get` and `List` keeps it to one place. Libraries like `sqlx` or `sqlc` reduce it further; learn the raw form first so you know what they're saving you.

Transactions

```
tx, err := s.db.BeginTx(ctx, nil)
if err != nil { return err }
defer tx.Rollback()           // no-op after Commit; guarantees cleanup on early
return
if _, err := tx.ExecContext(ctx, `UPDATE ...`); err != nil { return err }
if _, err := tx.ExecContext(ctx, `INSERT ...`); err != nil { return err }
return tx.Commit()
```

The `defer tx.Rollback()` idiom is the Go equivalent of a `with` block: every early `return err` rolls back, and the final `Commit` makes the rollback a harmless no-op.

The store interface pays off

Because Project 1 and Project 3 define `Store` as an interface in the consumer package, `SQLiteStore` is a drop-in replacement for `MemStore`. Run the same test suite against both, using `t.TempDir()` for the database file. Keep `MemStore` for handler tests (fast, no I/O) and test `SQLiteStore` in its own package against a real temporary file. Nothing in the `api` package changes when the storage changes; that is the payoff of Chapter 8's advice.

Self-check

1. Why is `sql.Open` not enough to know the database is reachable?
2. Why `defer rows.Close()` and check `rows.Err()`?
3. Explain `defer tx.Rollback()` followed by `return tx.Commit()` to someone who thinks it rolls back the commit.

CHAPTER 20

Configuration, logging and shipping one binary

The last mile of a service is unglamorous and where most tutorials stop. This chapter covers the small set of practices that make a Go service pleasant to run: explicit configuration, structured logs, a health endpoint, and a build that produces one file.

Configuration: flags with environment fallback

```
type Config struct {
    Addr    string
    DBPath  string
    Dev     bool
}

func loadConfig(args []string) (Config, error) {
    var cfg Config
    fs := flag.NewFlagSet("shelf", flag.ContinueOnError)
    fs.StringVar(&cfg.Addr, "addr", envOr("SHELF_ADDR", ":8080"), "listen address")
    fs.StringVar(&cfg.DBPath, "db", envOr("SHELF_DB", "shelf.db"), "sqlite database path")
    fs.BoolVar(&cfg.Dev, "dev", false, "serve templates from disk")
    if err := fs.Parse(args); err != nil {
        return Config{}, err
    }
    return cfg, nil
}

func envOr(key, def string) string {
    if v := os.Getenv(key); v != "" {
        return v
    }
    return def
}
```

Flags for humans, environment variables for containers and deployment, sensible defaults for both. Load configuration once in `main`, validate it, and pass the struct (or its fields) to whatever needs them. No configuration library is needed at this size.

Structured logging with `log/slog`

```
opts := &slog.HandlerOptions{Level: slog.LevelInfo}
logger := slog.New(slog.NewJSONHandler(os.Stdout, opts))
logger.Info("server starting", "addr", cfg.Addr, "db", cfg.DBPath)
logger.Error("query failed", "err", err, "item_id", id)

reqLog := logger.With("request_id", id) // child logger with fixed attributes
```

Key-value pairs after the message become JSON fields, so logs are searchable in any log system. Use `NewTextHandler` in development if you prefer readable lines. Pass the logger explicitly into the structures that need it (`Server{log: logger}`); the package-level `slog.Info` is convenient but

hides the dependency. Log at the edges (request middleware, background job outcomes, start-up and shutdown), not inside every function; the error wrapping from Chapter 7 carries the context up to the edge.

Health and readiness

Add `GET /healthz` that returns 200 and, if you like, checks `db.PingContext`. Every deployment platform and load balancer expects it, and it is the first thing you `curl` after starting.

The main that wires it together

```
func main() {
    if err := run(context.Background(), os.Args[1:]); err != nil {
        fmt.Fprintln(os.Stderr, "shelf:", err)
        os.Exit(1)
    }
}

func run(ctx context.Context, args []string) error {
    cfg, err := loadConfig(args)
    if err != nil { return err }
    logger := slog.New(slog.NewJSONHandler(os.Stdout, nil))

    ctx, stop := signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)
    defer stop()

    db, err := store.Open(ctx, cfg.DBPath)           // opens, pings, migrates
    if err != nil { return err }
    defer db.Close()

    srv := api.NewServer(store.NewSQLite(db), logger, cfg.Dev)
    httpSrv := &http.Server{
        Addr:           cfg.Addr,
        Handler:         srv.Routes(),
        ReadHeaderTimeout: 5 * time.Second,
    }

    errCh := make(chan error, 1)
    go func() { errCh <- httpSrv.ListenAndServe() }()
    logger.Info("listening", "addr", cfg.Addr)

    select {
    case err := <-errCh:
        return err                                // failed to bind, etc.
    case <-ctx.Done():
        logger.Info("shutting down")
        shutdownCtx, cancel := context.WithTimeout(context.Background(), 10*time.Second)
        defer cancel()
        return httpSrv.Shutdown(shutdownCtx)
    }
}
```

Read it top to bottom: configuration, logger, signal-aware context, database, server, then block until either the server fails or a signal arrives. This `main` is nearly identical in every Go service you will ever write, so learn it as a unit.

Building and running

```
go build -o bin/shelf ./cmd/shelf          # one file, templates and static assets included
./bin/shelf -addr :9000 -db data/shelf.db
GOOS=linux GOARCH=amd64 go build -o bin/shelf-linux ./cmd/shelf
```

Add a `Makefile` or a `Taskfile` with `build`, `test`, `run` targets if you like, but notice how little there is to script. There is no `node_modules`, no `virtualenv`, no runtime version to match on the target machine.

Self-check

1. Why does `run` take a context and return an error instead of `main` doing everything?
2. What is the difference between the `ctx` passed to `run` and the `shutdownCtx` created inside it, and why can't the second be derived from the first?
3. Where in this service do errors get logged, and where do they get wrapped?

PROJECT 3

shelf, a reading-list API with a web UI

shelf • a REST API, an HTML/JS frontend and a SQLite database in one binary

Time box: 10–14 hours over 5–6 sessions · **Chapters needed:** 17–20 (plus everything before) ·

Dependencies: `modernc.org/sqlite`

You will exercise: `net/http` routing and handlers, JSON request/response conventions, middleware, `html/template`, `embed`, `database/sql`, migrations, the store-interface swap, `httptest`, graceful shutdown, `slog`, configuration

Why this project

This is the project you asked for: an API, wired to an HTML frontend, running locally, that you can see working in a browser. A reading list (links you want to read, with status and tags) is small enough to finish and useful enough to keep. It carries the design lesson of the whole book: the `Store` interface lets you build and test everything against memory first, then swap in SQLite without touching a handler.

What it looks like when finished

```
$ shelf -addr :8080 -db shelf.db
{"time":"...", "level":"INFO", "msg":"listening", "addr":":8080"}

$ curl -s -X POST localhost:8080/api/items \
  -d '{"url":"https://go.dev/doc/effective_go", "title":"Effective Go", "tags":["go"]}'
{"id":1, "url":"https://go.dev/doc/effective_go", "title":"Effective Go", "notes":"","status":"unread", "tags":["go"], "added_at":"2026-09-05T10:12:03Z"}

$ curl -s localhost:8080/api/items?status=unread | jq '[][.title]'
"Effective Go"

$ curl -s -X PATCH localhost:8080/api/items/1 -d '{"status":"done"}'
{"id":1, "...", "status":"done", "finished_at":"2026-09-05T10:15:41Z"}

$ curl -s -X POST localhost:8080/api/items -d '{"url":"not a url"}'
{"error":"invalid url: must be absolute http(s)"} # HTTP 400

$ curl -si localhost:8080/api/items/999 | head -1
HTTP/1.1 404 Not Found
```

Opening `http://localhost:8080/` in a browser shows the list rendered by the server, with a form to add an item and a button per item to cycle its status, both talking to the JSON API from `app.js`.

API contract

METHOD & PATH	REQUEST	RESPONSE
GET /api/items?status=&tag=	–	200 array of items, newest first
POST /api/items	{url, title?, notes?, tags?}	201 item; 400 on validation error
GET /api/items/{id}	–	200 item; 404
PATCH /api/items/{id}	any subset of {title, notes, status, tags}	200 updated item; 400; 404
DELETE /api/items/{id}	–	204; 404
GET /healthz	–	200 {"status": "ok"}
GET /	–	200 HTML page
GET /static/*	–	embedded assets

Status is one of `unread`, `reading`, `done`; moving to `done` sets `finished_at`, moving away clears it. `title` defaults to the URL when omitted. Errors are always `{"error": "..."} with the right status code, and internal errors never expose details.`

Suggested layout

```
shelf/
  go.mod
  cmd/shelf/main.go
  internal/shelf/      item.go (Item, Status, validation, ListFilter)
                       errors.go (ErrNotFound, ValidationError)
  internal/store/      store.go (interface + shared test suite), memory.go,
                       sqlite.go, migrate.go, migrations/*.sql
  internal/api/         server.go (Server, Routes), handlers.go, middleware.go,
                       json.go, handlers_test.go
  web/                  templates/index.html, static/app.js, static/style.css, embed.go
  LEARNING.md
```

Milestones

MI Domain, memory store and read endpoints (2 h)

Define `Item`, `Status`, `ListFilter`, validation (absolute http(s) URL; status in the allowed set; title ≤ 200 chars), and the `Store` interface with context-taking methods. Implement `MemStore` (mutex-guarded map). Build `Server` with `Routes()` returning a mux, and implement `GET /api/items`, `GET /api/items/{id}`, `GET /healthz` with the `writeJSON`/`writeError` helpers.

Accept when: `httptest` tests cover list (empty and populated, status filter), get (found, not found, non-numeric id → 400), and `healthz`. `go run ./cmd/shelf` starts and `curl` agrees with the tests.

M2 Write endpoints, middleware, error mapping (2.5 h)

Implement `POST`, `PATCH`, `DELETE` with dedicated request structs (pointer fields for `PATCH`), `decodeJSON` with size limit and unknown-field rejection, and a single `writeStoreError(w, err)` that maps `ErrNotFound` → 404, `*ValidationError` → 400 with the message, everything else → 500 with a log line. Add `recoverPanic`, request logging with status code capture (wrap `ResponseWriter`), and a request ID header.

Accept when: table-driven handler tests cover each status code path; a deliberately panicking test route returns 500 and the server survives; a request with an unknown JSON field returns 400; logs show method, path, status and duration for every request.

M3 HTML page and JavaScript client (2 h)

Embed `web/`. Serve `GET /` from `index.html` via `html/template` (list, add form, status button per item, error area). Write `app.js`: submit the form via `fetch` to `POST /api/items`, cycle status via `PATCH`, delete via `DELETE`, refresh the list after each. Add a `-dev` flag that serves templates and static files from disk instead of the embedded copy.

Accept when: you can add, advance and delete items in the browser with the server running; a test for `GET /` asserts a 200, the HTML content type, and that an item's title appears escaped (add an item titled `<b>x</b>` and check the response contains `<b>x</b>`).

M4 SQLite store and migrations (3 h)

Implement `SQLiteStore` with the same interface, a `migrate(ctx, db, fs)` that applies numbered embedded SQL files once, and `store.Open(ctx, path)` that opens, configures pragmas, pings and migrates. Turn the `MemStore` tests into a shared suite `RunStoreTests(t, func() Store)` and run it against both implementations, the SQLite one against `t.TempDir()`.

Accept when: the shared suite passes for both stores; restarting the server preserves data; running migrations twice is a no-op; `List` with a status filter uses SQL, not Go-side filtering.

M5 Configuration, logging, graceful shutdown, build (1.5 h)

Wire the `main` from Chapter 20: flags with `env` fallback, `slog` JSON logger, `signal.NotifyContext`, `http.Server` with timeouts, `Shutdown` with a deadline, `db.Close()` on exit. Build `bin/shelf` and run it from a different directory to prove the assets are embedded.

Accept when: Ctrl-C during a slow request (add a test-only `GET /api/slow` that sleeps 3 s) lets the request finish and then exits cleanly; the binary runs from `/tmp` with no other files present; `go vet ./...` and `go test -race ./...` are clean.

Definition of done

- ☐ Every row of the API contract has at least one test, and the browser UI performs all three operations.

- ☐ The `api` package imports `shelf` and its own `Store` interface but never imports `store`; only `main` knows which store is in use.
- ☐ No handler contains SQL; no store contains HTTP.
- ☐ Internal error details never reach a client; every 500 has a corresponding log line with the wrapped error.
- ☐ You can explain the request lifecycle from `ListenAndServe` through middleware, mux, handler, store and back, including which goroutine each step runs on.
- ☐ Commit tagged `done`; `LEARNING.md` updated; optional AI review only after this point.

Stretch goals (optional, pick one)

- On `POST` with no title, fetch the page (reusing your Project 2 client with a 3 s timeout) and extract `<title>`; fall back to the URL on any failure and never let it block the response for long.
- Pagination with `?limit=&cursor=` and a `Link` header.
- Export/import: `GET /api/export` returns all items as JSON; `POST /api/import` accepts the same shape inside a transaction.
- Replace `location.reload()` with DOM updates from the API responses.

PART V

Capstone and roadmap

One project that combines everything, a week-by-week schedule with checkpoints, and an honest guide to what lies beyond the 20% and when to go there.

Project 4 · pulse, a local uptime monitor with a live dashboard

Chapter 21 · The six-week roadmap

Chapter 22 · Beyond the 20%

PROJECT 4 · CAPSTONE

pulse, a local uptime monitor with a live dashboard

pulse · background workers, persistence, an API and a live-updating page, shutting down cleanly as one process

Time box: 10–14 hours over 5–6 sessions · **Chapters needed:** all · **Dependencies:** modernc.org/sqlite; optionally golang.org/x/sync/errgroup

You will exercise: ticker loops with cancellation, bounded concurrency, mutex-guarded state, the HTTP client and server together, SQLite, server-sent events, graceful shutdown of several subsystems, tests for time-based code

Why this project

Everything you have built so far runs *in response* to something: a command, a request. Real services also do work on their own schedule, and coordinating background work with a web server, a database and a clean shutdown is the skill that separates “knows Go syntax” from “can ship a Go service”. An uptime monitor is the smallest program that has all of those parts and shows its results visibly, in a browser, in real time.

What it does

You register targets (name, URL, check interval, expected status). **pulse** checks each one on its interval with a timeout, records every result, keeps the current state of each target in memory, detects transitions (up → down, down → up) and logs them, and serves a dashboard that updates live without refreshing. It stores history in SQLite, prunes old results, and stops cleanly on Ctrl-C, finishing in-flight checks first.

```
$ pulse -addr :8080 -db pulse.db
{"level":"INFO","msg":"listening","addr":":8080"}
{"level":"INFO","msg":"scheduler started","targets":3}
{"level":"WARN","msg":"target down","name":"docs","url":"https://go.dev/doc",
  "status":0,"err":"context deadline exceeded"}
{"level":"INFO","msg":"target up","name":"docs","after":"2m10s"}
^C
{"level":"INFO","msg":"shutting down","in_flight_checks":2}
{"level":"INFO","msg":"stopped"}
```

The dashboard at `/` shows a row per target: a green/red state, the last latency, uptime over the last 24 hours, and the time since the last change, all updating as checks complete.

Architecture

one process · one binary · one root context from `signal.NotifyContext`

Scheduler one ticker goroutine per target · semaphore limits concurrent checks

State mutex-guarded map of current status per target + transition broadcaster

HTTP server JSON API · dashboard page GET `/api/events` streams state changes (SSE)

SQLite via Store interface targets · checks · retention job

results events

Three subsystems share one context and one store. The scheduler produces results; state records and broadcasts them; the server reads state and streams events.

Suggested layout

```
pulse/
  cmd/pulse/main.go
  internal/pulse/      target.go, check.go (Result, Checker with *http.Client),
                        state.go (State, Broadcaster)
  internal/scheduler/  scheduler.go, scheduler_test.go
  internal/store/      store.go, sqlite.go, migrate.go, migrations/
  internal/api/        server.go, handlers.go, events.go (SSE), middleware.go, *_test.go
  web/                templates/dashboard.html, static/app.js, static/style.css
```

Milestones

M1 Checker and scheduler, in memory (3 h)

`Checker.Check(ctx, target) Result` reuses your Project 2 code (status, latency, error, `OK` computed against the expected status). `Scheduler.Run(ctx, targets, onResult)` starts one goroutine per target running Chapter 15's Pattern 4 with that target's interval, bounds simultaneous checks with a semaphore, and returns only after all goroutines have exited when `ctx` is cancelled. Make the checker an interface so tests can inject a fake that returns scripted results.

Accept when: a test with a fake checker and 20 ms intervals asserts that after 100 ms each target has been checked at least 3 times and at most 6; cancelling the context stops the scheduler within one interval; `go test -race` is clean; `runtime.NumGoroutine()` returns to baseline.

M2 State, transitions and persistence (2.5 h)

`State` holds the latest `Result` per target under a mutex and exposes `Apply(Result)` (changed `bool`) and `Snapshot()`. On a change it logs and publishes an `Event{TargetID, Up bool, At}` to a `Broadcaster` (a mutex-guarded set of subscriber channels; publishing must never block on a slow subscriber: use a buffered channel per subscriber and drop on full). Persist every result to a `checks` table; load targets from a `targets` table at start-up; add a retention goroutine that deletes checks older than `-retain` (default 7 days) every hour.

Accept when: the shared store test suite passes against SQLite; a test subscribes two listeners, applies up→down→up, and receives exactly two events each; a slow subscriber does not delay `Apply` (assert with a timing bound).

M3 API and dashboard with live updates (3 h)

API: CRUD on `/api/targets` (adding a target must start checking it without a restart: the scheduler needs `Add/Remove`, each managing that target's goroutine), `GET /api/status` (snapshot with uptime % over 24 h computed in SQL), `GET /api/targets/{id}/checks?limit=50`. Dashboard: server-rendered table from the snapshot. `GET /api/events`: set `Content-Type: text/event-stream`, subscribe, and in a loop `select` on the subscriber channel and `r.Context().Done()`, writing `data: {...}\n\n` and calling `Flush()` (assert w. `(http.Flusher)`). `app.js` opens an `EventSource` and patches the row that changed.

Accept when: adding a target in the UI makes its row appear and start updating; stopping a local test server you're monitoring turns its row red within one interval, without a page refresh; closing the browser tab ends the SSE handler (log it) and unsubscribes.

M4 Shutdown, hardening, and running it for real (2 h)

On signal: stop accepting new HTTP connections, cancel the scheduler and wait for in-flight checks, close SSE streams, flush the last results, close the database, exit 0, all within a deadline. Add `-check-timeout`, `-max-concurrent`, and start-up validation of targets. Run `pulse` against five real sites and your own `shelf` server for a full day.

Accept when: a test starts the whole thing (via `run(ctx, args)` with a temp DB and `:0` for a random port), cancels the context, and asserts it returns within 2 s with no error and no leaked goroutines; after a day of running, the database contains the expected number of checks and the dashboard's uptime numbers make sense.

Definition of done

- ☐ All milestones pass under `go test -race ./...`; `go vet` is clean.
- ☐ You can draw every goroutine the process runs in steady state and name what stops each one.
- ☐ The scheduler, state, store and API are separately testable, and the tests prove it.
- ☐ Start-up and shutdown are logged, ordered, and bounded by deadlines.
- ☐ You have written, in `LEARNING.md`, one page on what you would change about the architecture now that it works.
- ☐ Commit tagged `done`. This is the point at which you can honestly say you understand Go deeply enough to build services without a model.

Stretch goals (optional)

- Webhook alerts on transitions with retry and backoff (`POST` a JSON payload to a configured URL).
- Per-target latency sparkline in the dashboard from the last 50 checks.
- A `pulse check URL` subcommand that reuses the checker for a one-off check (`flag` subcommands from Project 1).
- Replace the polling `Scheduler` with a single timer heap that fires the next due check (a real scheduler design; compare complexity in `LEARNING.md`).

CHAPTER 21

The six-week roadmap

This is the schedule the book was designed around: roughly eight to ten hours a week for six weeks, with reading always immediately followed by building. Adapt it, but keep the ordering and the checkpoints.

Week by week

WEEK	READ	BUILD	CHECKPOINT (END OF WEEK)
1	Part I (Ch 1–4), Ch 5–6	Toolchain set up; <code>taskr</code> M1–M2	Both stores pass the shared tests. You can explain the slice diagram from memory.
2	Ch 7–10	<code>taskr</code> M3–M4, tests, install and use it	<code>taskr</code> tagged <code>done</code> . You can write a table-driven test without looking.
3	Ch 11–16	<code>linkcheck</code> M1–M4	<code>linkcheck</code> tagged <code>done</code> under <code>-race</code> . You can draw the worker-pool shutdown sequence.
4	Ch 17–18	<code>shelf</code> M1–M3 (memory store, API, HTML page)	Browser UI works end to end against the in-memory store.
5	Ch 19–20	<code>shelf</code> M4–M5 (SQLite, shutdown, binary); rebuild <code>taskr</code> M1–M3 from the spec in 90 minutes	<code>shelf</code> tagged <code>done</code> . Rebuild completed with fewer than three doc lookups.
6	Re-read Ch 15–16	<code>pulse</code> M1–M4	<code>pulse</code> tagged <code>done</code> , running against real targets for a day.

If your time is irregular: milestone mode

Ignore the weeks. Move through the milestones in order (`taskr` M1 → M4, `linkcheck` M1 → M4, `shelf` M1 → M5, `pulse` M1 → M4), reading each chapter just before the first milestone that needs it. The only fixed rule: a session shorter than 60 minutes is for reading and self-checks, never for starting a milestone; and the rebuild of `taskr` happens at least a week after you finished it, wherever you are.

Session structure that works

1. **Two minutes:** write yesterday's key idiom from memory (Chapter 4). Compare; note the gap.
2. **Five minutes:** read the milestone's acceptance criteria again and write the first test you'll make pass.
3. **The bulk:** build, in test-first or test-soon loops. Commit at every green test.
4. **Five minutes:** `LEARNING.md` entry. What confused you, what clicked, one question to answer tomorrow.

Signals that you are ready to move on

Each chapter ends with self-check questions. Answer them out loud or in `LEARNING.md` before starting the milestone that depends on the chapter. If you can't answer one, re-read that section only; do not re-read the whole chapter. Each project's definition of done includes an "explain without notes" item; treat it as a real exam.

Signals that you are going too fast

You copy a fragment from the book into your project without retyping it. You skip a self-check. You look at your old `taskr` during the rebuild. You open a chat window "just to check". Each of these is a small breach of the contract from the introduction; none is fatal, but notice them and correct course. The goal is not to finish the book; it is to finish it *as someone who can build these things alone*.

After the six weeks

Pick one of the stretch goals you skipped, or one small tool you genuinely want (a Markdown link checker for your notes, a local file-sharing server, a webhook receiver that logs to SQLite) and build it from a blank directory with the same rules. When that takes you an evening rather than a week, you have what you came for.

CHAPTER 22

Beyond the 20%

Everything in Tier B and C of Chapter 1 is still waiting. Here is what each is for, when you will actually need it, and where to learn it. Do not study any of it speculatively; let a real problem pull you in.

Topics and when they become relevant

TOPIC	YOU NEED IT WHEN	START WITH
Iterators (<code>iter.Seq</code> , <code>range-over-func</code> , Go 1.23+)	You're writing a library that yields sequences lazily	<code>go doc iter</code> ; the <code>slices</code> and <code>maps</code> iterator functions you already use
Generic methods (Go 1.27)	A generic operation belongs conceptually to a type	The Go blog post "Generic Methods"
<code>encoding/json/v2</code> (Go 1.27)	You hit v1's performance or strictness limits	<code>go doc encoding/json/v2</code> ; v1 keeps working
Profiling with <code>pprof</code>	You have a measured performance or memory problem	<code>go test -cpuprofile</code> , <code>net/http/pprof</code> , the Go blog's profiling articles
Reflection (<code>reflect</code>)	You're writing a serializer, validator or ORM-like tool	<code>go doc reflect</code> ; read how <code>encoding/json</code> uses it
<code>go:generate</code> , <code>sqlc</code> , <code>stringer</code>	Boilerplate is provably slowing you down	<code>go help generate</code> ; <code>sqlc</code> docs for typed SQL
gRPC and protobuf	Two Go (or polyglot) services need typed RPC	The official gRPC Go quick start
Build constraints, <code>cgo</code> , <code>unsafe</code>	Platform-specific code or C interop	<code>go help buildconstraint</code> ; avoid <code>unsafe</code> unless you can explain the GC implications
Structured concurrency helpers (<code>errgroup</code> , <code>singleflight</code> , <code>semaphore</code>)	Recurring patterns from Chapter 15 need a battle-tested version	golang.org/x/sync
Testing helpers (<code>testing/synctest</code> , fuzzing)	Time-dependent or input-parsing code is hard to test deterministically	<code>go doc testing/synctest</code> ; <code>go help testfuzz</code>
Deployment (Docker, <code>systemd</code> , cross-compiling)	Someone other than you runs your service	A scratch or distroless Docker image with your static binary; it is a two-line Dockerfile

Reading that rewards the time

Books and long-form:

- **The Go Programming Language**, Donovan & Kernighan. Older (pre-generics) but the clearest explanation of the core language. Chapters 8 and 9 on concurrency are still the best available.

- **Learning Go** (2nd edition), Jon Bodner. Modern, covers generics and idioms, opinionated in the right ways.
- **100 Go Mistakes and How to Avoid Them**, Teiva Harsanyi. Read after Project 2. It will confirm and extend Chapter 16.
- **Let's Go** and **Let's Go Further**, Alex Edwards. Web services with the standard library, in far more depth than Part IV.

Official material, all free, all authoritative:

- The Tour (go.dev/tour) for a fast pass over syntax; Effective Go (go.dev/doc/effective_go) once you've finished `taskr`; the Go blog (go.dev/blog), especially “Go Concurrency Patterns”, “Pipelines and cancellation”, “Errors are values”, and “Slices: usage and internals”.
- pkg.go.dev and [go doc](https://go.dev/doc). Reading the standard library's documentation *and source* is the single most effective way to become fluent. Start with `strings`, `errors`, `sync`, `net/http`, `encoding/json`.
- Go by Example (gobyexample.com) for a runnable snippet of any construct.
- The Go Proverbs (go-proverbs.github.io), Rob Pike's compression of the culture into a dozen lines. Re-read them after each project; they will mean more each time.

On using AI again

Once you can build the four projects unaided, the fear you started with has been answered. The question then is not whether to use models but how to keep the skill you rebuilt. A rule that works: use a model for what you already know how to do (boilerplate you could write, code you can review line by line) and never for what you don't (a new library, a new concurrency shape, a bug you haven't understood). Keep one project at any time that is model-free, the way a musician keeps practicing scales. The competence you now have is not fragile if you use it.

A last word on the 80/20

You now have the 20%. The remaining 80% of Go is not more difficult; it is more *specific*, and you will learn each piece in an afternoon when you need it, because you understand the model it rests on. That is the whole bet of this book. Ship something.

APPENDICES

Reference

Material to keep open while you build: the idioms on two pages, a decoder for the error messages you will meet, and the checklists that tell you whether you are done.

Appendix A · The idiom cheat sheet

Appendix B · Error message decoder

Appendix C · Self-assessment checklists

APPENDIX A

The idiom cheat sheet

Every fragment here appears somewhere in the book. Print these two pages; when you can reproduce all of them from memory, you no longer need them.

Program skeleton

```
func main() {
    if err := run(context.Background(), os.Args[1:], os.Stdout); err != nil {
        fmt.Fprintln(os.Stderr, "prog:", err)
        os.Exit(1)
    }
}
func run(ctx context.Context, args []string, out io.Writer) error {
    ctx, stop := signal.NotifyContext(ctx, os.Interrupt, syscall.SIGTERM)
    defer stop()
    // ...
    return nil
}
```

Errors

```
var ErrNotFound = errors.New("not found") // sentinel
type ValidationError struct{ Field, Msg string } // typed
func (e *ValidationError) Error() string { return e.Field + ": " + e.Msg }

if err != nil { return fmt.Errorf("load %s: %w", path, err) } // wrap with context
if errors.Is(err, ErrNotFound) { } // sentinel test, through
wrapping
var ve *ValidationError
if errors.As(err, &ve) { } // extract a typed error
```

Data

```
type T struct{ ID int `json:"id"`; Name string `json:"name,omitempty"` }
func (t *T) Rename(n string) { t.Name = n } // pointer receiver mutates

s := make([]int, 0, 16); s = append(s, 1) // always reassign append
for i := range s { s[i]++ } // mutate in place by index
c := slices.Clone(s); slices.Sort(c)
m := map[string]int{}; v, ok := m["k"]; delete(m, "k")
for _, k := range slices.Sorted(maps.Keys(m)) { } // sorted map keys (Go
1.23+)
var sb strings.Builder; fmt.Fprintf(&sb, "%d", 1); _ = sb.String()
```

Interfaces

```
type Store interface { Get(ctx context.Context, id int64) (Item, error) } // consumer-
defined
var _ Store = (*SQLiteStore)(nil) // compile-time
check
func render(w io.Writer, items []Item) error { /* accept the narrowest interface */ }
switch v := x.(type) { case string: _ = v; case fmt.Stringer: _ = v.String() }
```

Concurrency

```
var wg sync.WaitGroup
for _, u := range urls { wg.Add(1); go func() { defer wg.Done(); work(u) }() }
wg.Wait()

jobs := make(chan Job); results := make(chan Result)
for range n { go worker(jobs, results) } // worker: for j := range jobs { results <-
do(j) }
go func() { defer close(jobs); for _, j := range all { jobs <- j } }()
go func() { wg.Wait(); close(results) }() // coordinator closes results
for r := range results { }

sem := make(chan struct{}, 10); sem <- struct{}{}; <-sem // semaphore acquire / release

type Cache struct { mu sync.Mutex; m map[string]string }
func (c *Cache) Get(k string) (string, bool) {
    c.mu.Lock(); defer c.mu.Unlock(); v, ok := c.m[k]; return v, ok
}

ctx, cancel := context.WithTimeout(ctx, 5*time.Second); defer cancel()
select { case v := <-ch: _ = v; case <-ctx.Done(): return ctx.Err() }

t := time.NewTicker(d); defer t.Stop()
for { select { case <-t.C: tick(); case <-ctx.Done(): return } }
```

HTTP

```

mux := http.NewServeMux()
mux.HandleFunc("GET /api/items/{id}", s.handleGet)           // id := r.PathValue("id")
mux.Handle("GET /static/", http.FileServerFS(webFS))

func writeJSON(w http.ResponseWriter, status int, v any) {
    w.Header().Set("Content-Type", "application/json"); w.WriteHeader(status)
    _ = json.NewEncoder(w).Encode(v)
}
dec := json.NewDecoder(http.MaxBytesReader(w, r.Body, 1<<20)); dec.DisallowUnknownFields()

func mw(next http.Handler) http.Handler {
    return http.HandlerFunc(func(w http.ResponseWriter, r *http.Request) {
        next.ServeHTTP(w, r)
    })
}
srv := &http.Server{Addr: ":8080", Handler: h, ReadHeaderTimeout: 5 * time.Second}
go func() { _ = srv.ListenAndServe() }(); <-ctx.Done(); _ = srv.Shutdown(shutdownCtx)

req := httptest.NewRequest(http.MethodGet, "/api/items/1", nil)
rec := httptest.NewRecorder()
mux.ServeHTTP(rec, req)                                     // rec.Code, rec.Body

```

SQL

```

db, _ := sql.Open("sqlite", "file:app.db"); _ = db.PingContext(ctx)
err := db.QueryRowContext(ctx, `SELECT a, b FROM t WHERE id = ?`, id).Scan(&a, &b)
if errors.Is(err, sql.ErrNoRows) { return ErrNotFound }
rows, _ := db.QueryContext(ctx, `SELECT ...`); defer rows.Close()
for rows.Next() { _ = rows.Scan(&a, &b) }; _ = rows.Err()
tx, _ := db.BeginTx(ctx, nil); defer tx.Rollback(); /* ... */ _ = tx.Commit()

```

Tests

```

tests := []struct{ name string; in string; want int; wantErr bool }{ /* rows */ }
for _, tc := range tests {
    t.Run(tc.name, func(t *testing.T) {
        got, err := f(tc.in)
        if (err != nil) != tc.wantErr {
            t.Fatalf("err = %v, wantErr %v", err, tc.wantErr)
        }
        if got != tc.want { t.Errorf("got %v, want %v", got, tc.want) }
    })
}
dir := t.TempDir()                                         // run: go test -race -count=1 -run TestName -v ./...

```

APPENDIX B

Error message decoder

Go's compiler and runtime messages are precise. Read them literally. Here are the ones you will see in your first month, what they mean, and the usual fix.

Compile-time

MESSAGE	MEANING	FIX
declared and not used: x	A local variable is never read	Delete it, use it, or name it <code>_</code>
"os" imported and not used	Unused import	Remove it; let <code>goimports</code> manage imports
undefined: foo	No such identifier in scope	Typo, missing import, unexported name from another package, or wrong package qualifier
cannot use x (variable of type string) as int value in argument to f	Type mismatch; Go never converts implicitly	Convert explicitly or fix the type
invalid operation: a + b (mismatched types int and int64)	Arithmetic across numeric types	<code>int64(a) + b</code>
assignment mismatch: 2 variables but f returns 1 value	Wrong number of results on the left	Match the function's signature; check <code>go doc</code>
no new variables on left side of :=	<code>:=</code> with only existing variables	Use <code>=</code>
missing return	A function with results can reach its end without returning	Add a final <code>return</code> (often after a <code>switch</code> without <code>default</code>)
x.Foo undefined (type T has no field or method Foo)	Field/method doesn't exist on that type	Check spelling and capitalization; you may have a value where you need a pointer or vice versa
cannot assign to struct field m[k].f in map	Map values aren't addressable	Copy out, modify, store back, or store pointers
T does not implement I (method M has pointer receiver)	Only <code>*T</code> has the method	Pass <code>&t</code> or construct with <code>&T{}</code>
T does not implement I (missing method M)	A method is missing or has the wrong signature	Compare signatures exactly, including parameter and result types
first argument to append must be a slice / cannot use append(...) (value of type []T) as ...	Misuse of <code>append</code>	<code>s = append(s, x)</code>

MESSAGE	MEANING	FIX
invalid argument: index 3 out of bounds [0:3]	Constant index too large for an array	Fix the index or use a slice
unexpected newline, expected comma or }	A multi-line literal is missing a trailing comma	Add the comma to the last element
syntax error: unexpected else, expected }	else on its own line	} else { on the same line, always
cannot use nil as T value	Assigning nil to a value type	Use the zero value, or make the field a pointer
go: cannot find main module	No go.mod in this or a parent directory	go mod init <path>
no required module provides package X	Import not in go.mod	go get X or go mod tidy
missing go.sum entry for module	Checksum not recorded	go mod tidy
the cancel function returned by context.WithTimeout should be called (vet)	Leaked context	defer cancel()
copylocks: ... passes lock by value (vet)	A Mutex inside a struct was copied	Pointer receivers; pass pointers

Run-time

MESSAGE	MEANING	FIX
panic: runtime error: index out of range [5] with length 3	Slice/array index too large	Check len; off-by-one in a loop bound
panic: runtime error: invalid memory address or nil pointer dereference	Method or field access through a nil pointer, or a nil interface	Find what wasn't initialized; check constructors and error paths that return nil with err == nil
panic: assignment to entry in nil map	Writing to a map you never made	m := map[K]V{} or make in the constructor
panic: interface conversion: interface {} is string, not int	Type assertion without , ok failed	Use v, ok := x.(int) or a type switch
fatal error: all goroutines are asleep - deadlock!	Every goroutine is blocked	A send with no receiver, a Wait with a missing Done, an unclosed channel in a range (Chapter 16)
panic: send on closed channel	Producer closed too early or a receiver closed	Only the last sender closes
panic: close of closed channel	Two owners	One owner, or sync.Once

MESSAGE	MEANING	FIX
<code>fatal error: concurrent map writes / concurrent map read and map write</code>	Unsynchronized map	Mutex in the owning struct
<code>panic: sync: negative WaitGroup counter</code>	More Done than Add	One Add per goroutine before go, one deferred Done inside
WARNING: DATA RACE (with -race)	Two goroutines accessed the same memory without synchronization	Read both stack traces; guard with a mutex or hand off via channel
<code>http: superfluous response.WriteHeader call</code>	<code>WriteHeader</code> twice, or after writing body	Return after writing an error; set status once
<code>http: panic serving 127.0.0.1:...</code>	A handler panicked; net/http recovered this connection only	Fix the bug; add the <code>recoverPanic</code> middleware
<code>context deadline exceeded / context canceled</code>	The operation's context expired or was cancelled	Expected; check with <code>errors.Is(err, context.DeadlineExceeded)</code> and decide what to do
<code>dial tcp ...: connection refused</code>	Nothing is listening at that address	Wrong port, server not started, or the server exited early
<code>database is locked</code> (SQLite)	Concurrent writers	<code>SetMaxOpenConns(1)</code> or a busy timeout pragma; WAL mode

APPENDIX C

Self-assessment checklists

Check each box only when you can do the thing from memory, without the book open. Anything unchecked at the end of a part is what you re-read before the next project.

Part I • Orientation

- ☐ Create a module, write a `main`, run it, build it, cross-compile it.
- ☐ Use `go doc` to look up a function's signature and a type's methods.
- ☐ State the eight mental-model shifts and give one line of Go for each.
- ☐ Recite the contract: what is allowed while building, what happens after `done`.

Part II • The core language

- ☐ Explain `:=` vs `=`, shadowing, and zero values, with a bug caused by each.
- ☐ Write an enum with `iota` and a `String()` method.
- ☐ Write a function with multiple returns and a variadic parameter; call it with a spread slice.
- ☐ Use all four `for` shapes and an expression-less `switch`.
- ☐ Explain `defer` ordering and argument evaluation time.
- ☐ Draw the slice header; predict the result of an `append` on a shared sub-slice with and without spare capacity.
- ☐ Explain why `range` over a slice of structs yields copies and give both fixes.
- ☐ Write map lookup with `ok`, delete, deterministic iteration; explain the nil-map panic.
- ☐ Explain bytes vs runes and predict `len("é")`.
- ☐ Choose value vs pointer receiver for a given type and justify it.
- ☐ Write a sentinel error, a typed error, wrap with `%w`, and check with `Is` and `As`.
- ☐ State when `panic` is acceptable.
- ☐ Define an interface at the consumer, satisfy it with two types, add the compile-time assertion.
- ☐ Write a function taking `io.Writer` and test it with a `bytes.Buffer`.
- ☐ Explain the typed-nil-in-interface gotcha and the pointer-receiver method-set gotcha.
- ☐ Write a generic `Map` and a `Max` with `cmp.Ordered`; say when not to use generics.
- ☐ Write a table-driven test with subtests from memory; run one subtest by name.

Part III • Concurrency

- ☐ Start goroutines and wait for them with a `WaitGroup`, with `Add` in the right place.
- ☐ Explain what the scheduler does and why blocking is cheap.
- ☐ Explain unbuffered vs buffered channels and when to use each.
- ☐ State the closing rules and who closes with multiple senders.
- ☐ Write a `select` with a timeout and a cancellation case; explain the `time.After`-in-a-loop leak.
- ☐ Guard a map with a mutex inside its struct; explain why the snapshot is a clone.
- ☐ Explain `sync.Once`, `atomic`, and the channel-vs-mutex decision table.

- ☐ Use `context`: first parameter, `defer cancel()`, `Done` in loops, `NotifyContext`.
- ☐ Write the worker pool (Pattern 1) from memory and narrate its shutdown.
- ☐ Write a semaphore, an `errgroup` with a limit, a ticker loop and a pipeline stage.
- ☐ Read a race detector report and find the two access sites.
- ☐ Diagnose the deadlock message, a goroutine leak, and a send on a closed channel.

Part IV • Services

- ☐ Write handlers on a server struct with dependencies; route with method-and-wildcard patterns.
- ☐ Write `writeJSON`, `writeError`, `decodeJSON` and explain each safeguard.
- ☐ Write logging and recovery middleware and compose them in the right order.
- ☐ Configure `http.Server` timeouts and implement graceful shutdown.
- ☐ Test a handler with `httptest.NewRecorder` and a full server with `httptest.NewServer`.
- ☐ Render an `html/template` into a buffer; explain contextual escaping.
- ☐ Embed assets and serve them; switch to disk in development.
- ☐ Write `Query` / `QueryRow` / `Exec` with placeholders, `Scan`, `rows.Err`, and a transaction with deferred rollback.
- ☐ Apply embedded migrations once and idempotently.
- ☐ Load config from flags with env fallback; log with `slog` key-value pairs.
- ☐ Write the canonical `main` / `run` for a service from memory.

Part V • Capstone

- ☐ Design and draw the goroutine topology of a service with background workers before writing code.
- ☐ Broadcast events to many subscribers without blocking the producer.
- ☐ Stream server-sent events from a handler and stop when the client disconnects.
- ☐ Shut down several subsystems in order, within a deadline, with no leaked goroutines.
- ☐ Test time-based code with injected fakes and short intervals under `-race`.
- ☐ Build, from a blank directory and with no model, a small service you actually use.